

УЧЕБНИК ДЛЯ ТЕХ, КТО
НИКОГДА НЕ ПРОГРАММИРОВАЛ

Сайт с нуля

Как собрать интернет-магазин
и стать fullstack-разработчиком

60 глав · практика на живом коде

autodoc.tj

Содержание

- | | |
|----------|--|
| 1 | Что такое сайт и что мы построим |
| 2 | Как работает интернет |
| 3 | Инструменты: редактор, терминал, браузер |
| 4 | Первый файл на экране |
| 5 | Теги и структура документа |
| 6 | Текст, ссылки, картинки, списки |
| 7 | Таблицы и формы |
| 8 | Практика: карточка товара и каталог |

Книга пишется по частям. Здесь — главы, готовые на сегодня.

Что такое сайт и что мы построим

ЗАЧЕМ ЭТО

Зачем эта глава

Здравствуйте. Если вы открыли эту книгу, скорее всего, вы уже пробовали подступиться к программированию — и что-то пошло не так. Так бывает почти у всех, и дело обычно не в вас.

Смотрите, что происходит на типичных курсах. Первые две недели вам рассказывают про «переменные», «циклы» и «типы данных». Всё это правда нужно. Но вы сидите и думаете: *а где, собственно, сайт?* Знания вроде появляются, а куда их приложить — непонятно. Это как учиться водить, изучая устройство карбюратора: полезно, но до машины вы так и не дошли.

Поэтому первая глава — не про код вообще. Она про **карту местности**.

Мы разберёмся, из каких частей состоит любой сайт, кто эти части делает и как они между собой связаны. И когда в главе 20 появятся те самые переменные, вы уже будете точно знать, в каком месте магазина они живут и зачем нужны.

Никакого кода набирать пока не надо. Просто читайте — как будто вам показывают дом перед тем, как дать в руки инструменты.

РАЗБИРАЕМСЯ

Начнём с обычного магазина

Забудьте на минуту про компьютеры. Правда, совсем забудьте.

Представьте магазин автозапчастей на вашей улице. Тот самый, куда вы заходили за щётками стеклоочистителя. Что там есть?

Во-первых, торговый зал. Полки, товар, ценники, витрина у входа, касса. Всё разложено так, чтобы покупатель разобрался сам: колодки — сюда, фильтры — туда, акция — на видном месте. Здесь красиво и понятно. Покупатель дальше этой зоны не заходит.

Во-вторых, подсобка. Дверь с табличкой «Посторонним вход воспрещён». Там принимают товар, считают выручку, оформляют накладные, решают, кому дать скидку, а кому нет. Там же лежат деньги. Покупателю туда нельзя — и это не из вредности, а просто потому, что иначе цены на ценниках начнут переписывать все кому не лень.

В-третьих, складской журнал. Толстая тетрадь или программа на компьютере, где записано: какой товар, сколько штук, почём куплено, почём продаём, кто привёз, кому отгрузили. Скучная штука. Но потеряйте её — и магазин превратится в склад коробок, про которые никто ничего не знает.

Всё. Три зоны. Держите эту картинку в голове — сейчас будет главное.

► Сайт устроен ровно так же

Не «похоже». Не «примерно». А один в один:

В МАГАЗИНЕ НА УЛИЦЕ	НА САЙТЕ	КАК ЭТО ЗОВУТ ПРОГРАММИСТЫ
Торговый зал	То, что видно в браузере	Фронтенд (frontend — «передняя часть»)
Подсобка	Программа на сервере	Бэкенд (backend — «задняя часть»)
Складской журнал	База данных	БД (database)

Когда вы заходите на любой сайт — маркетплейс, банк, доставку еды — вы всегда стоите в торговом зале. Всё остальное происходит за дверью, куда вас не пускают.

А человек, который умеет работать **во всех трёх зонах**, называется **fullstack-разработчик**. Дословно — «полный набор». Именно им вы и станете к концу книги.

► Ещё немного про эти три зоны

Чтобы совсем закрепилось, вот та же тройка на других примерах.

Ресторан. Зал со столиками и меню — фронтенд. Кухня, где готовят и считают себестоимость, — бэкенд. Список продуктов на складе и книга заказов — база данных.

Поликлиника. Регистратура и коридор с креслами — фронтенд. Кабинет врача, где принимают решения, — бэкенд. Картотека с медкартами — база данных.

Такси в телефоне. Карта с машинками и кнопка «Заказать» — фронтенд. Программа, которая ищет ближайшего водителя и считает цену, — бэкенд. Хранилище поездок, водителей и оценок — база данных.

Заметили? Везде одно и то же: **красивая витрина, скрытая от глаз логика и место, где всё записано.** Любой сайт, любое приложение, любой сервис.

РАЗБИРАЕМСЯ

Пять инструментов, и всё

Хорошая новость: чтобы построить магазин, нужно не пятьдесят технологий. Нужно **пять**. Давайте познакомимся с каждой — с примерами, чтобы не осталось тумана.

► HTML — скелет

HTML отвечает на вопрос «**что здесь есть?**». Не «как выглядит», а именно «что есть».

Он говорит браузеру: тут заголовок, тут абзац текста, тут картинка, тут кнопка, тут список из трёх пунктов, тут таблица. И всё — на этом его работа заканчивается. Красотой он не занимается принципиально.

Примеры того, что делает HTML:

- на странице появился заголовок «Тормозные колодки»;
- под ним — абзац с описанием;
- слева — фотография товара;
- ниже — список характеристик: производитель, артикул, вес;
- в конце — кнопка «Купить».

Аналогия: скелет человека. Есть череп, рёбра, руки, ноги. На свидание в таком виде не пойдёшь, но сразу понятно: это человек, вот голова, вот конечности.

Ещё точнее — **чертёж дома**. На чертеже нет обоев и мебели, но видно: тут комната, тут окно, тут дверь.

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

Это уже настоящая, рабочая страница. Уродливая до слёз — но рабочая. Браузер её откроет.

► CSS — внешность

CSS отвечает на вопрос «**а как это выглядит?**».

Тот же самый скелет, но теперь у него появились цвет, шрифт, размер, отступы, тени, скруглённые углы и понятное расположение на экране.

Примеры того, что делает CSS:

- кнопка стала красной, а буквы на ней белыми;
- заголовок — крупный и жирный, описание — помельче и посветлее;
- между карточками товаров появились ровные промежутки;
- товары выстроились в три колонки вместо кучи;
- на телефоне те же товары встали в одну колонку;
- при наведении мышкой карточка чуть приподнялась.

Аналогия: одежда, причёска и осанка. Скелет тот же, а впечатление совсем другое.

Ещё точнее — **ремонт в квартире**. Планировка не изменилась: те же комнаты, те же окна. Но после ремонта это другое жильё.

```
button { background: #d92b2b; color: white; border-radius: 8px; }
```

Одна строчка — и серая системная кнопка превратилась в красную кнопку с мягкими углами.

► JavaScript — рефлексы браузера

JavaScript отвечает на вопрос «что произойдёт, если человек что-то сделает?» — причём прямо здесь, в браузере, без перезагрузки страницы.

Примеры того, что делает JavaScript:

- нажали «Купить» — счётчик корзины мгновенно стал «3»;
- набрали в поиске «коло» — снизу выпали подсказки: «колодки», «кольцо», «коллектор»;
- кликнули на фото — оно открылось на весь экран;
- на телефоне нажали три полоски — выехало меню;
- начали заполнять форму и забыли телефон — поле подсветилось красным ещё до отправки;
- пролистали страницу вниз — шапка «прилипла» к верху экрана.

Аналогия: рефлекс. Дотронулись до горячего — рука отдёрнулась сама, мозг ещё даже не понял, что случилось.

Ещё точнее — **автоматические двери в магазине**. Никто не звонит директору, чтобы вам открыли. Датчик увидел человека — двери разъехались.

```
button.onclick = () => alert('Товар добавлен в корзину');
```

► PHP — мозг на сервере

А вот это самое важное место, и здесь нужно быть внимательным.

PHP работает **не в браузере**. Он живёт на **сервере** — на чужом компьютере, который стоит где-то в дата-центре и работает круглосуточно. Покупатель до него не дотягивается в принципе.

PHP принимает все серьёзные решения.

Примеры того, что делает PHP:

- проверяет, правильный ли пароль ввели при входе;
- решает, показать человеку кнопку «Админка» или не показывать;
- считает итоговую цену: цена поставщика + наценка + доставка;
- применяет скидку постоянному покупателю — или не применяет;
- записывает заказ и присваивает ему номер;
- отправляет письмо «ваш заказ принят»;
- не даёт продавцу Ахмаду редактировать товары продавца Сафара.

Аналогия: мозг. Или директор магазина — тот, кто решает, а не тот, кто улыбается в зале.

А теперь — правило, которое стоит запомнить прямо сейчас.

JavaScript выполняется **на компьютере покупателя**. Это значит, что покупатель может его открыть, почитать и переписать. Это не сложно — любой браузер позволяет.

PHP выполняется **на вашем сервере**. Покупатель его не видит и изменить не может.

Представьте, что цена товара считается на JavaScript. Тогда покупатель открывает инструменты браузера, меняет 250 на 1 — и оформляет заказ на сомони. Это как если бы в магазине разрешили покупателям самим переклеивать ценники.

Поэтому **цены, скидки, доступы и всё, что связано с деньгами, считает только PHP.**

Никогда — JavaScript. Мы к этому ещё вернёмся не раз, но услышать это лучше в первой главе.

```
if ($user['role'] === 'admin') {  
    echo 'Добро пожаловать в админку';  
}
```

► SQL — разговор с памятью

SQL — это язык, на котором разговаривают с базой данных. Спрашивают и записывают.

Примеры того, что делает SQL:

- найти все тормозные колодки дешевле 300 сомони, которые есть в наличии;
- достать данные одного товара по его номеру;
- добавить новый заказ;
- изменить статус заказа с «новый» на «отправлен»;
- посчитать, на какую сумму продал каждый продавец за месяц;
- найти покупателей, которые не заходили полгода.

Аналогия: разговор с кладовщиком. Вы не идёте на склад сами и не роетесь в коробках. Вы говорите: «дай мне все колодки Bosch, что есть в наличии» — и он приносит список.

```
SELECT name, price FROM products WHERE price < 300 AND in_stock = 1;
```

► Коротко, чтобы уложилось

ИНСТРУМЕНТ	ОТВЕЧАЕТ НА ВОПРОС	ГДЕ РАБОТАЕТ	БЫТОВАЯ АНАЛОГИЯ
HTML	Что здесь есть?	В браузере	Скелет, чертёж дома
CSS	Как это выглядит?	В браузере	Одежда, ремонт
JavaScript	Что будет, если нажать?	В браузере	Рефлексы, автодвери
PHP	Что решаем и что показываем?	На сервере	Мозг, директор
SQL	Где это записано?	В базе данных	Кладовщик, картотека

Если сейчас каша в голове — это абсолютно нормально. Никто не запоминает пять новых понятий с первого раза. Мы к каждому вернёмся отдельной главой, и не по одному разу.

Выше промелькнуло пять кусочков кода, и они наверняка выглядели как шифр. Давайте расшифруем — **каждый** значок.

Запоминать ничего не нужно. Просто прочитайте один раз, чтобы код перестал пугать. Это как впервые посмотреть на ноты: сначала мурашки, потом оказывается, что там всего семь букв.

► HTML: почему всё в «уголках»

```
<h1>Тормозные колодки</h1>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
< и >	«уголки», угловые скобки	Внутри уголков — команда браузеру , а не текст для чтения. На экране их не будет
<h1>	«эйч-один»	Заголовок первого уровня , самый главный на странице. От английского <i>heading</i> — заголовок. Бывают <h1> ... <h6> : чем больше цифра, тем мельче заголовок
</h1>	«закрывающий эйч-один»	Косая черта / означает конец . «Заголовок закончился вот здесь»
<p>	«пи»	Абзац обычного текста. От <i>paragraph</i>
<button>	«баттон»	Кнопка , на которую можно нажать
Текст между тегами		То единственное, что реально увидит человек на экране

Пара «открывающий тег + содержимое + закрывающий тег» называется **элемент**.

Правило простое, как в математике: **что открыли — то надо закрыть**. Забыли закрыть — браузер начнёт фантазировать, и страница поедет.

А что такое <div> ?

Это слово вы будете встречать чаще всех остальных, поэтому знакомимся сразу.

`<div>` — это **пустая коробка**. Он ничего не означает и никак не выглядит. На экране его не видно вообще.

Зачем тогда нужен? Чтобы **сложить в него другие элементы** и потом обращаться ко всей группе сразу.

```
<div>
  <h1>Тормозные колодки</h1>
  <p>Цена: 250 сомони</p>
  <button>Купить</button>
</div>
```

Теперь заголовок, цена и кнопка лежат в одной коробке. И можно сказать: «вот эту коробку сделай белой, добавь рамку, скругли углы и отодвинь от соседних». Получится **карточка товара** — та самая, из которых собран любой каталог.

Название `div` — сокращение от английского *division*, «раздел», «отделение». Думайте о нём как о коробке из-под обуви: сама по себе не нужна, но сложить в неё вещи и подписать — очень удобно. Подробно разберём в главе 5.

► CSS: почему фигурные скобки

```
button { background: #d92b2b; color: white; border-radius: 8px; }
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
<code>button</code>	селектор	Кого красим. Здесь — все кнопки на странице
<code>{ }</code>	фигурные скобки	«Начало и конец списка указаний». Всё внутри относится к кнопке
<code>background</code>	свойство	Что меняем — цвет фона
<code>:</code>	двоеточие	Разделитель: слева что меняем, справа на что
<code>#d92b2b</code>	значение	Цвет в виде кода: <code>#</code> и шесть символов. Здесь — красный
<code>;</code>	точка с запятой	Конец одного указания. Дальше пойдёт следующее
<code>color: white</code>		Цвет текста — белый. Не путайте с <code>background</code> : это фон
<code>border-radius: 8px</code>		Скруглить углы на 8 пикселей

Формула CSS всегда одна и та же:

кого `{` что меняем `:` на что `;` `}`

Выучите её один раз — и любой CSS в мире станет читаемым.

► JavaScript: точки и стрелки

```
button.onclick = () => alert('Товар добавлен в корзину');
```

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
<code>button</code>		Кнопка, которую мы до этого нашли на странице
<code>.</code>	точка	«Возьми у этого вот это». Как «Иван ^{***} . ^{***} телефон» — телефон Ивана
<code>onclick</code>	«он-клик»	Свойство «что делать при нажатии». <i>on click</i> — «по щелчку»
<code>=</code>	равно, присваивание	Положи в левое то, что справа. Это не «равенство» из математики, а команда «запиши»
<code>() =></code>	«стрелочная функция»	«Вот действие , которое надо выполнить». В круглых скобках — место для входных данных, здесь они не нужны
<code>alert(...)</code>	«алерт»	Готовая команда браузера: показать всплывающее окошко
<code>'...'</code>	кавычки	Внутри кавычек — текст , а не команда. Компьютер не пытается его выполнить
<code>;</code>	точка с запятой	Конец строки-команды

► PHP: доллары и условия

```
if ($user['role'] === 'admin') {
    echo 'Добро пожаловать в админку';
}
```

Вот здесь появилось `if` — пожалуй, самое важное слово во всём программировании. Оно есть в каждом языке, и работает везде одинаково.

ЧТО ВИДИМ КАК ЧИТАЕТСЯ ЧТО ЭТО ЗНАЧИТ

<code>if</code>	«иф», по-английски «если»	Проверка. «Если условие верно — сделай то, что в фигурных скобках. Если нет — просто пропусти»
<code>()</code>	круглые скобки после <code>if</code>	Внутри — условие , которое проверяем
<code>\$</code>	доллар	В PHP так помечают переменную — коробочку с данными. <code>\$user</code> — «коробочка с пользователем»
<code>\$user['role']</code>		Достать из коробочки <code>\$user</code> поле <code>role</code> — роль. Квадратные скобки: «возьми вот эту часть»
<code>===</code>	«строго равно»	Сравнение: «а точно ли слева то же самое, что справа?» Три знака, не один
<code>'admin'</code>		Текст, с которым сравниваем
<code>{ }</code>	фигурные скобки	Тело условия — что делать, если проверка прошла
<code>echo</code>	«эхо»	Команда PHP: выведи это на страницу
<code>;</code>		Конец команды

Один `=` или три `===` ? Самая частая ошибка новичка.

- `=` — это **записать**. `$x = 5` значит «положи пятёрку в коробочку x».
- `===` — это **спросить**. `$x === 5` значит «а в коробочке правда пятёрка?»

Первое — действие, второе — вопрос. Перепутаете — программа вместо проверки молча запишет значение, и ошибку вы будете искать долго.

А где же `else` ?

`else` — «иначе» — это вторая половина `if`. Читается ровно как в жизни:

```
if ($user['role'] === 'admin') {
    echo 'Добро пожаловать в админку';
} else {
    echo 'Вам сюда нельзя';
}
```

Дословно: **если** роль администратор — покажи приветствие, **иначе** — откажи. Одно из двух выполнится обязательно, третьего не дано.

Вы пользуетесь этой конструкцией каждый день, просто не называете её так:

- **если** дождь — беру зонт, **иначе** — не беру;
- **если** денег хватает — покупаю, **иначе** — откладываю;
- **если** на светофоре зелёный — иду, **иначе** — стою.

Вот и всё программирование. Подробно — в главе 21.

► SQL: запрос как обычная фраза

```
SELECT name, price FROM products WHERE price < 300 AND in_stock = 1;
```

SQL специально сделали похожим на английское предложение. Читайте по кусочкам:

ЧТО ВИДИМ	КАК ЧИТАЕТСЯ	ЧТО ЭТО ЗНАЧИТ
SELECT	«селект» — выбрать	Какие колонки нам нужны
name, price		Название и цена. Запятая — «и ещё»
FROM	«фром» — из	Из какой таблицы берём
products		Таблица товаров
WHERE	«вэа» — где, при условии	Какие строки берём, а какие пропускаем
price < 300		Цена меньше 300
AND	«энд» — и	Оба условия должны выполняться одновременно
in_stock = 1		Есть в наличии (1 — да, 0 — нет)
;		Конец запроса

Целиком получается: «Выбери название и цену из таблицы товаров, где цена меньше 300 и товар в наличии». Практически человеческая фраза.

► Что общего у всех пяти языков

А теперь приятное. Присмотритесь к таблицам выше — значки-то повторяются:

ЗНАЧОК

ПОЧТИ В ЛЮБОМ ЯЗЫКЕ ОЗНАЧАЕТ

`{ }`

«блок»: вот начало группы, вот конец

`( )`

«сюда кладут данные»: условие или входные значения

`;`

«команда закончилась»

`' '` или `" "`

«внутри текст, выполнять его не надо»

`//` или `#`

комментарий: заметка для человека, компьютер её не читает

Разобравшись один раз, вы будете узнавать их везде — хоть в Python, хоть в Java, хоть в языке, который придумают через десять лет. Языков много, а привычки у них общие.

РАЗБИРАЕМСЯ

А теперь всё вместе

Давайте проследим, что происходит, когда покупатель открывает страницу товара.

Прочитайте медленно — это самая важная схема в книге, всё остальное будет крутиться вокруг неё.

1. Покупатель нажимает ссылку «Тормозные колодки».
2. Браузер отправляет на сервер запрос: «дайте мне страницу товара номер 42».
3. На сервере просыпается PHP и думает: «сорок второй, значит. Надо узнать подробности».
4. PHP составляет запрос на SQL и отправляет его в базу данных: `SELECT * FROM products WHERE id = 42`.
5. База отвечает: «Колодки тормозные, Bosch, 250 сомони, на складе 7 штук».
6. PHP берёт эти данные и строит из них HTML: название вставляет в заголовок, цену — в ценник, картинку — в блок картинки.
7. Готовый HTML улетает обратно в браузер.
8. Браузер читает HTML, применяет CSS — и страница из чёрно-белого текста превращается в нормальный магазин.
9. Подключается JavaScript и начинает следить за кнопкой «Купить».

Что происходит, когда покупатель открывает страницу товара

Три зоны, пять технологий — и меньше секунды на всё



Всё это заняло меньше секунды. Пять технологий, и каждая сделала ровно свою часть работы.

Разделение обязанностей — вот главная идея всей веб-разработки. HTML не красит. CSS не считает. JavaScript не лезет в базу. PHP не рисует. Каждый занят своим делом и не мешает соседям.

Звучит как формальность, но именно поэтому сайты вообще можно чинить. Сломалась внешность — идёте в CSS и точно знаете, что база данных тут ни при чём.

РАЗБИРАЕМСЯ

Что именно мы построим

Не «примерчик на три кнопки». Настоящий интернет-магазин — такой, каким пользуются живые люди и в котором ходят реальные деньги.

Для покупателя:

- каталог запчастей с поиском по названию и по артикулу;
- фильтры: бренд, цена, наличие;
- карточка товара с фото, описанием и характеристиками;
- корзина: добавить, изменить количество, удалить;
- оформление заказа с адресом и телефоном;
- личный кабинет с историей заказов и их статусами.

Для продавца — потому что магазин будет не только ваш, а маркетплейс:

- регистрация продавца и его страница;
- выкладка своих товаров;
- свои заказы и работа с ними;
- баланс: сколько заработано и сколько уже выплачено.

Для администратора:

- модерация: проверять товары продавцов до публикации;
- управление всеми заказами и статусами;
- расчёт комиссии магазина;
- реестр выплат продавцам.

И взрослые вещи, которых обычно нет в учебниках:

- подключение внешнего API поставщика запчастей;
- пересчёт цены из рублей в сомони по живому курсу, плюс наценка и доставка;
- защита от взлома: SQL-инъекции, XSS, кража сессии;
- выкладка сайта на настоящий хостинг, под настоящим доменом, с замочком HTTPS.

Дойти до конца — это работа на несколько месяцев. Но каждая глава заканчивается чем-то, что можно открыть в браузере и показать. Ощущение «у меня получилось» будет приходить регулярно, а не один раз в самом конце.

РАЗБИРАЕМСЯ

Почему PHP, а не что-то модное

Отвечу честно, потому что вопрос вы всё равно услышите. В интернете любят повторять, что «PHP устарел».

PHP выбран не потому, что он самый модный. А потому, что для **первого** сайта он самый удобный:

- **Результат виден сразу.** Создали файл, открыли в браузере — работает. Не нужно ставить пятьдесят пакетов и настраивать сборщик, чтобы вывести на экран слово «привет».
- **Дешёвый хостинг.** PHP работает на хостинге за 3–5 долларов в месяц. Для первого настоящего проекта это важно: сайт можно показать людям, не разорившись.
- **Огромная база готовых ответов.** Какую бы проблему вы ни встретили, её уже кто-то решил и описал.
- **На нём работает больше половины сайтов в мире.** Один только WordPress — это примерно четверть всего интернета.

Но главное вот что. **Навык — не в языке.**

Когда вы научитесь думать «пришёл запрос → проверили права → сходили в базу → собрали ответ», вы пересядете на Python или Node.js за пару недель. Синтаксис разный, мышление одно и то же.

Языки приходят и уходят. Умение раскладывать задачу на части остаётся.

ГРАБЛИ

Грабли

Четыре мысли, которые чаще всего мешают новичкам. Если поймаете себя на любой из них — вспомните эту страницу.

«**Сначала выучу всё, потом начну делать**». Не сработает, проверено многими. Знания без практики выветриваются за неделю. Правильный порядок обратный: делаем маленькую вещь → упираемся в незнание → узнаём ровно столько, сколько нужно прямо сейчас → делаем дальше. Скучно, зато держится.

«**Надо сразу выбрать самый лучший язык**». Лучшего языка не существует, есть подходящий для задачи. Споры «PHP против Python» для новичка — это спор о том, каким молотком забивать первый в жизни гвоздь. Забейте хоть каким, потом разберётесь.

«**Настоящие программисты знают всё наизусть**». Нет. Разработчик с пятнадцатью годами опыта гуглит синтаксис по десять раз в день, и это нормально. Ценность не в памяти, а в умении разбить задачу на части и понять, что именно искать.

«**Я не математик, значит, не потяну**». Для веб-разработки нужна арифметика уровня седьмого класса: сложить цены, посчитать проценты, поделить на количество. Всё. Высшая математика понадобится, если вы пойдёте в машинное обучение — но это совсем другая история.

Пять упражнений. Ни в одном не нужен компьютер с редактором — только голова и браузер. Не пропускайте: именно на задачах прочитанное превращается в понятное.

Задача 1.1. Откройте любой знакомый сайт — маркетплейс, новости, банк. Выпишите пять вещей, которые вы **видите** (это работа фронтенда). Потом пять вещей, которые происходят **невидимо**: проверка пароля, расчёт доставки, сохранение заказа (это бэкенд).

Задача 1.2. Для каждой строки определите, кто отвечает — HTML, CSS, JavaScript, PHP или SQL:

1. Кнопка стала синей вместо серой.
2. На странице появился второй заголовок.
3. При наборе букв в поиске выпадают подсказки.
4. Проверка, что пароль при входе правильный.
5. Достать из базы 20 самых дешёвых товаров.
6. Меню на телефоне схлопнулось в три полоски.
7. Заказ записался, и ему присвоился номер.
8. Между карточками товаров появились отступы.

Задача 1.3. Объясните вслух — правда вслух, это работает — своими словами разницу между фронтендом и бэкендом. Уложитесь в два предложения и не используйте сами слова «фронтенд» и «бэкенд».

Задача 1.4. Почему скидку постоянного покупателя нельзя считать на JavaScript? Ответ — одна строчка.

Задача 1.5. Нажмите на любом сайте правую кнопку мыши и выберите «Посмотреть код страницы». Откроется HTML, который прислал сервер. Найдите в нём заголовок страницы (ищите `<h1>` или `<title>`). Если сейчас это выглядит как сплошная каша — так и должно быть. К главе 8 вы будете читать её свободно.

Задача 1.6. Придумайте три своих аналогии для тройки «витрина — подсобка — журнал». Не из книги, а свои: школа, аэропорт, парикмахерская — что угодно. Чем больше своих примеров, тем прочнее держится в голове.

Ответы — в Приложении Г.

В БОЮ В бою

А теперь заглянем в настоящий проект. В корне этого репозитория лежит код работающего магазина autodoc.tj. Открывать файлы пока не нужно — просто посмотрите на названия папок. Вы уже можете угадать, что где:

ПАПКА	ЧТО ВНУТРИ	КАКАЯ ЭТО ЗОНА
assets/css/	Файлы оформления	Фронтенд (CSS)
pages/	Страницы сайта	Фронтенд + бэкенд
includes/	Общая логика, которую используют все страницы	Бэкенд (PHP)
sql/	Структура базы данных	База данных
admin/	Админка	Бэкенд, «подсобка»
seller/	Кабинет продавца	Бэкенд, «подсобка»
api/	Точки, куда обращается JavaScript	Стык фронта и бэка

Узнаёте? Это тот самый магазин из начала главы. Торговый зал, подсобка и складской журнал — только разложенные по папкам.

Через шестьдесят глав вы откроете любой из этих файлов и будете понимать, что там написано, строчка за строчкой.

- Сайт — это **торговый зал** (фронтенд), **подсобка** (бэкенд) и **складской журнал** (база данных). Всегда, у любого сайта.
- **HTML** — скелет, **CSS** — внешность, **JavaScript** — рефлекс в браузере, **PHP** — мозг на сервере, **SQL** — разговор с памятью.
- JavaScript выполняется у покупателя и переписывается им за минуту. PHP выполняется у вас. Поэтому деньги и доступы считает **только PHP**.
- `if` — это «если», `else` — «иначе». `=` записывает, `===` спрашивает. `<div>` — пустая коробка для группировки.
- **Fullstack** — тот, кто работает во всех трёх зонах. Это вы через шестьдесят глав.
- Учиться нужно от задачи к теории, а не наоборот.

В следующей главе разберёмся, что вообще происходит между моментом, когда вы вводите адрес сайта, и моментом, когда на экране появляется картинка. Спойлер: там интересно.

Как работает интернет

ЗАЧЕМ ЭТО**Зачем эта глава**

Вы вводите в браузере адрес, жмёте Enter — и через полсекунды видите страницу. Кажется, что там нечего объяснять: нажал и появилось.

На самом деле за эти полсекунды происходит примерно десять отдельных событий, и каждое из них однажды у вас сломается. Сайт не открывается, картинки не грузятся, форма отпавилась «в нукуда», на телефоне работает, а на компьютере нет.

Человек, который не понимает эту цепочку, в такие моменты просто разводит руками.

Человек, который понимает, спокойно спрашивает себя: «так, а на каком шаге сломалось?» — и находит причину за пять минут.

Поэтому потратим главу на то, что происходит между вашим Enter и картинкой на экране. Кода снова не будет. Будет понимание, без которого дальше всё превратится в магию.

РАЗБИРАЕМСЯ**Интернет — это почта**

Самое вредное представление об интернете — «где-то там облако, и в нём всё лежит». Давайте заменим его на правильное.

Интернет — это почтовая служба. Огромная, быстрая, но обычная почта.

Есть отправитель. Есть получатель с адресом. Есть письмо с вопросом и письмо с ответом. Есть курьеры, которые везут письма по маршруту. Всё, никакой магии.

Когда вы открываете сайт, происходит ровно это:

- 1.** Вы (браузер) пишете письмо: «здравствуйте, пришлите мне, пожалуйста, главную страницу».
- 2.** Письмо едет по адресу.
- 3.** Там его читает сервер и готовит ответ.
- 4.** Ответ приезжает обратно.
- 5.** Браузер разворачивает конверт и показывает содержимое.

Одно письмо туда, одно обратно. Это называется **запрос** и **ответ** (по-английски *request* и *response*). Запомните эту пару — она будет встречаться в книге сотни раз.

Что происходит между вашим Enter и картинкой на экране

Одно письмо туда, одно обратно — и десяток шагов между ними

- 1 Разбирает адрес** **БРАУЗЕР**
Делит `https://autodoc.tj/catalog` на протокол, домен и путь
- 2 Спрашивает номер дома** **DNS**
«Какой IP у autodoc.tj?» — «185.12.94.7». Справочник интернета
- 3 Стучится и договаривается о шифровании** **БРАУЗЕР**
Тот самый замочек в адресной строке появляется здесь
- 4 Отправляет запрос** **БРАУЗЕР**
`GET /catalog HTTP/1.1` — «покажи, пожалуйста, каталог»
- 5 Думает** **СЕРВЕР**
PHP идёт в базу через SQL, забирает товары и собирает HTML. 20–200 мс
- 6 Отвечает** **СЕРВЕР**
`200 OK` и готовый HTML. Или `404`, если такого адреса нет
- 7 Рисует страницу и просит остальное** **БРАУЗЕР**
За каждым стилем, скриптом и картинкой уходит отдельный запрос. Их бывает 30–80

Где обычно ломается: шаг 2 — домен куплен, но не настроен, сайт «не открывается». Шаг 5 — ошибка в вашем коде, приходит `500`. Шаг 6 — опечатка в адресе, приходит `404`.

РАЗБИРАЕМСЯ

Клиент и сервер

Два слова, которые нужно развести раз и навсегда.

Клиент — тот, кто просит. В нашем случае это **браузер**: Chrome, Safari, Firefox, Яндекс.Браузер — любой. Он живёт на вашем телефоне или компьютере.

Сервер — тот, у кого просят. Это компьютер, который включён круглосуточно и умеет отвечать на письма.

И вот тут первое открытие для многих: **сервер** — это обычный компьютер. Не волшебный ящик. У него есть процессор, память, диск и операционная система. Отличий всего три:

- он не выключается;
- у него нет монитора и клавиатуры — им управляют по сети;
- он стоит не дома, а в дата-центре, где хороший интернет и не пропадает электричество.

Ваш ноутбук тоже может стать сервером. Мы это сделаем уже в главе 4: запустим на нём маленький сервер и откроем собственный сайт. Никакой мистики.

Кстати, «клиент» и «сервер» — это роли, а не предметы. Один и тот же компьютер может быть сервером для одних и клиентом для других. Наш магазин будет сервером для покупателей и клиентом, когда пойдёт спрашивать цены у поставщика.

РАЗБИРАЕМСЯ

Что происходит, когда вы жмёте Enter

Теперь подробно. Вы набрали `autodoc.tj` и нажали Enter. Поехали.

► Шаг 1. Браузер разбирает адрес

Адрес сайта — это не одно слово, а несколько частей. Возьмём полный вид:

```
https://autodoc.tj/catalog/brakes?page=2
```

КУСОК	КАК НАЗЫВАЕТСЯ	ЧТО ЗНАЧИТ
<code>https</code>	протокол	На каком языке разговаривать. <code>https</code> — «по-человечески, но зашифрованно»
<code>://</code>	разделитель	Просто знак, что дальше идёт адрес
<code>autodoc.tj</code>	домен	Имя дома, куда едем
<code>/catalog/brakes</code>	путь	Какая именно комната в доме нужна
<code>?page=2</code>	параметры	Уточнение: «вторая страница»

Браузер это всё раскладывает и понимает, куда собирается.

► Шаг 2. Браузер узнаёт настоящий адрес — DNS

Вот тут интересное. Компьютеры не умеют находить друг друга по именам вроде `autodoc.tj`. Они работают с числами — IP-адресами, например `185.12.94.7`.

Имя нужно людям. Число нужно машинам.

Поэтому существует DNS — телефонный справочник интернета. Браузер спрашивает: «какой номер у `autodoc.tj`?» — и получает: «185.12.94.7».

Аналогия проще некуда. Вы же не помните номер телефона мамы — вы открываете контакты и жмёте «Мама». Контакты сами подставляют цифры. DNS — это контакты интернета.

⚠ Почему это важно знать: если сайт «не открывается», очень часто виноват именно DNS. Домен купили, но не настроили — и справочник просто не знает, куда вас отправить. В главе 58 мы это настроим руками.

► Шаг 3. Браузер стучится в дверь

Зная IP-адрес, браузер устанавливает **соединение** с сервером. Как позвонить и дождаться, пока снимут трубку.

Если адрес начинался с `https`, то в этот момент собеседники ещё и договариваются о шифровании: «давай дальше разговаривать так, чтобы посторонние не поняли». Именно поэтому в браузере появляется замочек.

Чем `https` отличается от `http` на бытовом уровне:

- `http` — открытка. Её может прочитать любой почтальон по дороге.
- `https` — запечатанный конверт. Видно, что письмо едет, но не видно, что внутри.

Пароли и номера карт по открыткам не отправляют. Поэтому сегодня `https` обязателен.

► Шаг 4. Браузер отправляет запрос

Наконец само письмо. Выглядит оно примерно так:

```
GET /catalog/brakes HTTP/1.1
Host: autodoc.tj
User-Agent: Chrome
Accept-Language: ru
```

Не пугайтесь, читается легко:

СТРОКА	ЧТО ГОВОРИТ
GET	«Дай посмотреть». Это метод — тип просьбы
/catalog/brakes	Что именно дать
HTTP/1.1	На какой версии языка говорим
Host:	В какой дом стучимся
User-Agent:	Кто пришёл: браузер представляется
Accept-Language:	«Я предпочитаю русский»

Строки после первой называются **заголовками** — это уточнения к просьбе.

Методов запроса всего **несколько**, и два из них вам понадобятся постоянно:

МЕТОД	СМЫСЛ	ПРИМЕР ИЗ ЖИЗНИ МАГАЗИНА
GET	«Покажи»	Открыть каталог, посмотреть товар
POST	«Прими и запиши»	Оформить заказ, отправить форму
PUT	«Замени целиком»	Обновить карточку товара
DELETE	«Удали»	Удалить товар из корзины

Разницу GET и POST разберём подробно в главе 24, когда будем делать формы. Пока запомните так: **GET смотрит, POST меняет**.

► Шаг 5. Сервер думает

Письмо дошло. Сервер его прочитал и понял: просят страницу каталога тормозных колодок.

И вот здесь просыпается всё то, о чём мы говорили в первой главе. Запускается **PHP**. Он идёт в **базу данных** с запросом на **SQL**, получает список товаров, подставляет их в шаблон и собирает **HTML**.

Занимает это обычно от 20 до 200 миллисекунд. Если дольше — пользователи начинают уходить, и в главе 30 мы будем разбираться, почему тормозит.

► Шаг 6. Сервер отвечает

Ответ выглядит так:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

<!DOCTYPE html>
<html>
  <head><title>Тормозные колодки</title></head>
  <body>... </body>
</html>
```

Сверху — служебная часть, снизу, после пустой строки, — сам HTML.

200 OK — это **код ответа**. Сервер всегда сообщает, чем закончилось дело. Вот те коды, которые вы будете видеть каждый день:

КОД	ЧТО ОЗНАЧАЕТ	КОГДА ВЫ ЕГО ВСТРЕТИТЕ
200	Всё хорошо, вот страница	Обычная работа
301 / 302	Переехали, идите по другому адресу	Перенаправление, например с <code>http</code> на <code>https</code>
403	Вижу вас, но не пущу	Полез в админку без прав
404	Такой страницы нет	Опечатка в адресе, удалённый товар
500	Сервер сам сломался	Ошибка в вашем коде. Самый частый гость новичка
503	Сервер перегружен	Слишком много посетителей разом

Запомните различие, оно экономит часы: **404** — **вы ошиблись адресом**. **500** — **ошиблись вы, но в коде**. Первое лечится ссылкой, второе — чтением логов (глава 56).

► Шаг 7. Браузер рисует страницу

HTML приехал. Браузер читает его сверху вниз и строит из него страницу.

Но HTML — это только текст. По дороге ему встречаются ссылки на другие файлы: стили, картинки, скрипты. И за каждым файлом браузер **отправляет отдельный запрос**.

Вот это открытие обычно удивляет: одна страница — это не одно письмо, а **десятки**.

Запрос 1 → сама страница (HTML)
Запрос 2 → файл стилей (CSS)
Запрос 3 → скрипт (JavaScript)
Запрос 4 → логотип
Запрос 5 → фото товара №1
Запрос 6 → фото товара №2
... и так далее

Обычная страница магазина — это 30-80 запросов. Поэтому сайты и «весят»: не текст тяжёлый, а картинки, которых много.

► Шаг 8. Оживает JavaScript

Последним включается JavaScript и начинает следить за действиями: клики, ввод, прокрутка. Страница из картинки превращается в живую.

Собрали много новых слов. Разложим по полочкам — это ваш словарь на все оставшиеся главы.

СЛОВО	КАК ЧИТАЕТСЯ	ЧТО ЭТО ПРОСТЫМИ СЛОВАМИ
HTTP	«эйч-ти-ти-пи»	Язык, на котором браузер разговаривает с сервером. Правила переписки
HTTPS	«эйч-ти-ти-пи-эс»	То же самое, но зашифрованное. S — от <i>secure</i> , «безопасный»
Запрос (request)		Письмо от браузера к серверу: «дай мне вот это»
Ответ (response)		Письмо обратно: «вот, держи» или «не могу, вот почему»
Клиент		Тот, кто просит. Обычно браузер
Сервер		Тот, у кого просят. Компьютер, который всегда включён
IP-адрес	«ай-пи»	Номер компьютера в сети: 185.12.94.7
Домен		Человеческое имя сайта: autodoc.tj
DNS	«ди-эн-эс»	Справочник, который переводит имя в номер
Хостинг		Услуга: вам сдают в аренду место на сервере
Заголовки (headers)		Уточнения к письму: кто я, что понимаю, на каком языке
GET	«гет»	Метод «покажи»
POST	«пост»	Метод «прими и запиши»
Код ответа		Число-итог: 200 — хорошо, 404 — нет такого, 500 — сломалось
Порт		Номер двери в доме. У http дверь 80, у https — 443

Про порт чуть подробнее, потому что он вам скоро встретится. Если IP-адрес — это дом, то порт — квартира в этом доме. На одном компьютере может работать несколько программ, и каждая слушает свою дверь. Когда в главе 4 мы запустим сервер на своём компьютере, адрес будет выглядеть как `localhost:8000` — вот `8000` и есть номер двери.

А `localhost` — это «мой собственный компьютер». Всегда. На любой машине. Его IP — `127.0.0.1`, и он тоже всегда один и тот же.

НА ЭКРАНЕ

Посмотрите на это своими глазами

Всё описанное можно увидеть прямо сейчас, без единой строчки кода.

1. Откройте любой сайт в Chrome.
2. Нажмите F12 (на Mac — `Cmd+Option+I`). Откроется панель разработчика.
3. Перейдите на вкладку **Network** («Сеть»).
4. Обновите страницу клавишей F5.

Вы увидите список — это все те самые запросы. Каждая строка: имя файла, код ответа, размер, сколько миллисекунд занял.

Пощёлкайте по строкам. Справа откроются заголовки запроса и ответа — те же самые, что мы разбирали выше, только настоящие.

Это не учебная симуляция. Это реальная переписка вашего браузера с сервером. Профессиональные разработчики смотрят в эту вкладку каждый день.

ГРАБЛИ

Грабли

«Сайт лежит» — а на самом деле нет. Прежде чем паниковать, откройте его в другом браузере, с телефона по мобильному интернету и в режиме инкогнито. Очень часто «лежит» только у вас: кэш, DNS, провайдер, старая версия страницы.

Кэш — ваш главный обманщик. Браузер запоминает файлы, чтобы не качать их дважды. Полезно для скорости, вредно при разработке: вы поправили стили, а браузер показывает старые. Лечится жёстким обновлением: `Ctrl+F5` (на Mac `Cmd+Shift+R`). Запомните эту комбинацию — она сэкономит вам десятки часов недоумения.

Путать 404 и 500. 404 — «я не нашёл такой адрес». 500 — «я нашёл, но при выполнении всё упало». Это разные проблемы и разные места поиска.

Думать, что **https** защищает сайт от взлома. Не защищает. **https** шифрует только дорогу между браузером и сервером. Если в вашем коде дыра, шифрование канала не поможет никак. Безопасностью займёмся в главе 36.

Считать, что домен и хостинг — одно и то же. Домен — это имя. Хостинг — это место, где живёт сайт. Покупаются отдельно, часто у разных компаний. Как отдельно взять номер телефона и отдельно снять квартиру.

ЗАДАЧИ

Задачи

Задача 2.1. Разберите адрес на части — назовите протокол, домен, путь и параметры:

`https://autodoc.tj/search?q=kolodki&brand=bosch`

Задача 2.2. Какой код ответа вы получите в каждом случае?

1. Открыли главную страницу работающего сайта.
2. Ошиблись в адресе на одну букву.
3. Программист забыл точку с запятой, и PHP упал.
4. Зашли в админку, не будучи администратором.

Задача 2.3. Откройте панель разработчика (F12 → Network) на любимом сайте и обновите страницу. Сколько всего запросов ушло? Какой файл оказался самым тяжёлым? Запишите числа — в главе 57 мы вернёмся к ним, когда будем ускорять свой сайт.

Задача 2.4. Объясните своими словами, что такое DNS. Одно предложение, без слова «DNS».

Задача 2.5. Что произойдёт, если DNS сломается, а сайт при этом работает? Сможете ли вы попасть на сайт вообще?

Задача 2.6. Почему форму оформления заказа отправляют методом POST, а не GET? Подсказка: подумайте, что окажется в адресной строке браузера.

Ответы — в Приложении Г.

В коде `autodoc.tj` эта глава встречается буквально везде.

Загляните в любой файл в папке `api/` — например `api/vin_order_request.php`. Первые строки почти каждого такого файла проверяют метод запроса:

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {  
    http_response_code(405);  
    exit;  
}
```

Дословно: «если пришли не методом POST — верни код 405 („метод не разрешён“) и заканчивай». Это защита: страницу, которая создаёт заказ, нельзя открывать простой ссылкой.

Через двадцать глав вы будете писать такие проверки не задумываясь.

- Интернет — это почта. **Запрос** туда, **ответ** обратно.
- **Клиент** просит (браузер), **сервер** отвечает. Сервер — обычный компьютер, который не выключают.
- DNS переводит имя `autodoc.tj` в номер `185.12.94.7`. Как контакты в телефоне.
- HTTP — язык переписки. **GET** смотрит, **POST** записывает.
- **Коды ответа**: 200 — хорошо, 404 — нет адреса, 500 — сломался код, 403 — не пушу.
- Одна страница — это десятки запросов: HTML, стили, скрипты, каждая картинка.
- F12 → **Network** показывает всю эту переписку вживую. Пользуйтесь.
- Ctrl+F5 обновляет страницу мимо кэша. Запомните прямо сейчас.

В следующей главе перестанем читать и наконец возьмём инструменты в руки.

Инструменты: редактор, терминал, браузер

ЗАЧЕМ ЭТО

Зачем эта глава

Хорошая новость: чтобы делать сайты, не нужен мощный компьютер и не нужны платные программы. Нужны три вещи, и все три бесплатные.

Плохая новость: одна из них — **терминал**, то самое чёрное окно с буквами. Именно на нём спотыкается больше всего новичков. Не потому, что он сложный, а потому, что выглядит недружелюбно.

В этой главе мы поставим всё необходимое и — главное — подружимся с терминалом. К концу главы вы будете уверенно перемещаться по папкам и запускать программы, не трогая мышку.

РАЗБИРАЕМСЯ

Три инструмента, и всё

ИНСТРУМЕНТ	ЧТО ДЕЛАЕТ	АНАЛОГИЯ ИЗ МАСТЕРСКОЙ
Редактор кода	В нём вы пишете код	Верстак, за которым работаете
Терминал	В нём вы запускаете программы	Пульт управления станками
Браузер	В нём вы смотрите результат	Зеркало: сразу видно, что получилось

Всё. Ни «мощного железа», ни платных подписок. Подойдёт любой компьютер, купленный за последние десять лет.

► Почему не Word и не «Блокнот»

Первый вопрос новичка: а нельзя писать код в Word? Нельзя, и вот почему.

Word — это **текстовый процессор**. Он не хранит просто буквы, он хранит оформление: шрифты, размеры, отступы. И когда вы сохраните файл, туда попадёт куча невидимого мусора. Браузер такое не поймёт.

Код — это **чистый текст**. Только символы, ничего больше.

«Блокнот» из Windows чистый текст умеет, но он совершенно ничем не помогает. Это как строить дом одним молотком: теоретически можно, практически — мучение.

► Что ставить: Visual Studio Code

Берите VS Code — бесплатный редактор от Microsoft. Он стал стандартом: на нём работают миллионы разработчиков, под него есть всё на свете.

Скачивается с code.visualstudio.com, ставится как обычная программа, работает на Windows, macOS и Linux.

► Что редактор делает за вас

Вот почему в нём в разы приятнее, чем в блокноте:

- **Подсвечивает синтаксис.** Теги одним цветом, текст другим, ошибки третьим. Глаз сам находит нужное место.
- **Закрывает скобки.** Написали `<div>` — редактор сам поставит `</div>`. Помните правило «что открыли, то закрой»? Он будет следить за этим вместо вас.
- **Подсказывает.** Начали писать `back` — предложит `background`, `background-color` и остальные варианты.
- **Показывает ошибки до запуска.** Забыли кавычку — подчеркнёт красным сразу.
- **Ищет по всему проекту.** Где у меня в тридцати файлах написано `price`? Одна команда.

► Три настройки, которые стоит сделать сразу

Откройте настройки (Ctrl+, или Cmd+,) и поищите по названию:

1. **Word Wrap** → включить (`on`). Длинные строки будут переноситься, а не уезжать за край экрана.
2. **Font Size** → 15 или 16. Глаза скажут спасибо, а вы просидите за кодом много часов.
3. **Auto Save** → `afterDelay`. Файл сохраняется сам. Половина «загадочных ошибок» у новичков — это несохранённый файл.

► Два расширения, которые пригодятся

В VS Code слева есть иконка с кубиками — это магазин расширений. Поставьте:

- **PHP Intelephense** — умные подсказки по PHP.
- **Russian Language Pack** — если хочется интерфейс на русском.

Остальное — потом, по мере надобности. Не увлекайтесь: новички часто ставят тридцать расширений вместо того, чтобы писать код.

РАЗБИРАЕМСЯ

Инструмент второй: терминал

Вот к нему и подошли. Давайте сразу снимем страх.

► Что это вообще такое

Терминал — это способ управлять компьютером словами, а не мышкой. Всё.

Вы уже умеете управлять мышкой: открыть папку, скопировать файл, запустить программу. В терминале вы делаете то же самое, но пишете команду.

Зачем, если мышкой привычнее? Четыре причины:

1. **Многие программы просто не имеют окон.** Тот же PHP-сервер запускается только командой.
2. **Быстрее.** Одна строка вместо десяти кликов.
3. **Повторяемо.** Команду можно записать, отдать коллеге, вставить в инструкцию.
4. **На сервере другого способа нет.** У сервера нет монитора — только терминал. Когда в главе 57 будем выкладывать сайт, работать придётся именно так.

► Где его найти

СИСТЕМА	КАК ОТКРЫТЬ
Windows	Пуск → напечатать PowerShell → Enter
macOS	Cmd+Пробел → напечатать Терминал → Enter
Linux	Ctrl+Alt+T
Прямо в VS Code	Меню Терминал → Новый терминал, или Ctrl+`

Последний способ — самый удобный. Терминал открывается прямо под кодом, и не нужно прыгать между окнами.

► Как читать эту непонятную строчку

Открыли терминал и увидели что-то вроде:

```
firdavs@MacBook ~/projects %
```

Расшифровываем:

ЧАСТЬ	ЧТО ЗНАЧИТ
firdavs	ваше имя пользователя
MacBook	имя компьютера
~/projects	где вы сейчас находитесь — самое важное
% или \$ или >	приглашение: «пиши команду»

Ключевая мысль: терминал всегда находится в какой-то папке. Как вы, открывая проводник, всегда стоите в какой-то папке. Команды выполняются относительно неё.

Значок ~ означает вашу домашнюю папку. На Windows это C:\Users\Firdavs, на Mac — /Users/firdavs.

► Семь команд, которых хватит надолго

Правда семь. Не сто, не тридцать — семь.

КОМАНДА	РАСШИФРОВКА	ЧТО ДЕЛАЕТ
<code>pwd</code>	<i>print working directory</i>	Где я сейчас? Показывает полный путь
<code>ls</code>	<i>list</i>	Что здесь лежит? Список файлов и папок
<code>cd имя</code>	<i>change directory</i>	Зайти в папку
<code>cd ..</code>		Подняться на уровень выше
<code>mkdir имя</code>	<i>make directory</i>	Создать папку
<code>touch имя</code>		Создать пустой файл
<code>clear</code>		Очистить экран, когда намусорили

На Windows в PowerShell работают все эти команды, кроме `touch`. Вместо него: `New-Item имя.txt`.

► Три приёма, которые экономят время

1. Клавиша Tab дописывает за вас. Начните печатать `cd proj` и нажмите Tab — терминал сам допишет `projects`. Это не только быстрее, но и защищает от опечаток.
2. Стрелка вверх достаёт прошлые команды. Нажмите ↑ — появится предыдущая команда. Ещё раз — позапрошлая.
3. Ctrl+C прерывает. Запустили что-то и хотите остановить — Ctrl+C. Работает почти везде. Понадобится, когда будем гасить наш сервер.

► Потренируйтесь прямо сейчас

Не читайте дальше — откройте терминал и наберите по очереди:

```
pwd
ls
mkdir moi-magazin
cd moi-magazin
pwd
ls
cd ..
```

Что произошло: посмотрели, где мы → посмотрели содержимое → создали папку → зашли в неё → убедились, что вошли → увидели, что она пустая → вышли обратно.

Всё. Вы только что поработали в терминале. Ничего страшного не случилось.

РАЗБИРАЕМСЯ

Инструмент третий: браузер

Браузер у вас уже есть. Но нужно узнать про него кое-что новое.

Возьмите **Chrome** или **Firefox** для разработки. Не потому, что они лучше как браузеры, а потому, что у них лучшие инструменты разработчика и вся документация написана под них.

► Панель разработчика — ваш второй дом

Нажмите F12 (на Mac **Cmd+Option+I**). Панель, которую вы уже открывали в прошлой главе. Вот вкладки, которые понадобятся:

ВКЛАДКА ДЛЯ ЧЕГО

Elements	Смотреть HTML страницы и менять его на лету . Можно поменять цвет кнопки и сразу увидеть результат
Console	Ошибки JavaScript. Первое место, куда смотреть, когда «ничего не работает»
Network	Все запросы: что загрузилось, что нет, сколько заняло
Sources	Исходные файлы страницы

Совет, который экономит нервы: приучитесь держать консоль открытой, когда пишете JavaScript. Половина проблем в ней прямым текстом объяснена — просто туда никто не смотрит.

Пока ставить не нужно, но чтобы вы знали, что впереди:

ЧТО	ЗАЧЕМ	В КАКОЙ ГЛАВЕ
PHP	Чтобы работал серверный код	Глава 19
MySQL	База данных	Глава 26
Git	Хранить историю изменений и не терять работу	Глава 53

Всё это ставится одним пакетом — **XAMPP** (Windows/Mac) или **OpenServer** (Windows). Один установщик кладёт сразу PHP, MySQL и веб-сервер. Подробную инструкцию дам в главе 19, когда дойдём до PHP, — чтобы не ставить впрок то, чем ещё не пользуемся.

Заведите порядок сразу, потом будете благодарны.

Создайте папку для всех своих проектов в домашней директории:

```
projects/  
  moi-magazin/      ← наш будущий магазин  
  eksperimenty/     ← пробы и опыты
```

Два правила, которые избавят от боли:

Никаких русских букв и пробелов в названиях файлов и папок. Не `Мой магазин/каталог товаров.php`, а `moi-magazin/catalog.php`. Веб-серверы, командная строка и системы контроля версий с русскими буквами и пробелами дружат плохо, и вы поймаете «мистическую» ошибку на ровном месте.

Не храните проект в облачных папках. Не кладите код в Dropbox, iCloud или «Рабочий стол», который синхронизируется. Облако начнёт синхронизировать тысячи мелких файлов и создавать конфликтные копии. Для истории изменений есть Git — им и займёмся в главе 53.

«Я боюсь что-то сломать в терминале». Команды из этой книги ничего не ломают. Опасны ровно две вещи: `rm -rf` (удаляет безвозвратно) и всё, что начинается с `sudo` (выполняется от имени администратора). Мы их не используем. Правило простое: **не вставляйте в терминал команду, которую не понимаете**, особенно из случайных статей в интернете.

Файл сохранён? Точно? Классика жанра: правите код, обновляете браузер, ничего не меняется. В VS Code несохранённый файл помечен **точкой** рядом с именем вкладки. Включите автосохранение и забудьте про эту проблему.

Расширение файла имеет значение. `index.html` — страница. `index.txt` — просто текст, браузер покажет исходник. На Windows расширения по умолчанию скрыты, и легко создать `index.html.txt`, не заметив этого. Включите показ расширений: Проводник → Вид → «Расширения имён файлов».

Кириллица в коде. В самом тексте страницы русский, конечно, можно и нужно. Нельзя — в именах файлов, папок и названиях переменных.

Задача 3.1. Установите VS Code и сделайте три настройки из главы: перенос строк, размер шрифта, автосохранение.

Задача 3.2. Откройте терминал и выполните цепочку. Запишите, что вывела каждая команда:

```
pwd
mkdir projects
cd projects
mkdir moi-magazin
cd moi-magazin
pwd
```

Задача 3.3. Найдите на своём компьютере домашнюю папку через `pwd`, а потом откройте её же в обычном проводнике. Убедитесь, что это одно и то же место. Важно почувствовать: терминал и проводник — два окна в один и тот же дом.

Задача 3.4. Откройте любой сайт, нажмите F12 → вкладка **Elements**. Найдите заголовок страницы, дважды кликните по тексту и замените его на своё имя. Нажмите Enter.

Что произошло на самом деле — вы взломали сайт? Ответ обязательно посмотрите в приложении, он важный.

Задача 3.5. Откройте F12 → **Console** на трёх разных сайтах. Найдите хотя бы один с красными ошибками. Их полно даже у крупных компаний — это нормально и полезно знать.

Задача 3.6. Создайте на компьютере папку `projects/moi-magazin` — там мы будем строить магазин всю оставшуюся книгу.

Ответы — в Приложении Г.

В БОЮ

Откройте в проводнике корень этого репозитория и посмотрите на имена файлов:

`seller_shop.php` , `product_edit.php` , `AutoEuroPriceProvider.php` .

Заметьте: **ни одного русского слова, ни одного пробела**. Только латиница, и слова разделены подчёркиванием. Это не эстетство — это то самое правило, которое избавляет от необъяснимых сбоев на боевом сервере.

ИТОГ

Итог

- Нужны три инструмента: **редактор** (VS Code), **терминал**, **браузер** (Chrome). Все бесплатные.
- Код — это **чистый текст**. Word и «Блокнот» не годятся.
- Терминал — управление компьютером словами. Он всегда стоит в какой-то папке.
- Хватит **семи команд**: `pwd`, `ls`, `cd`, `cd ..`, `mkdir`, `touch`, `clear`.
- **Tab** дописывает, **стрелка вверх** повторяет, **Ctrl+C** прерывает.
- **F12** открывает панель разработчика: Elements, Console, Network.
- В именах файлов — **только латиница, без пробелов**.
- Не держите проект в облачной папке.

В следующей главе делаем первую страницу и открываем её в браузере. Наконец-то руками.

Первый файл на экране

ЗАЧЕМ ЭТО**Зачем эта глава**

Три главы мы читали. Хватит.

Сейчас вы сделаете страницу, откроете её в браузере и увидите на экране то, что написали своими руками. Это займёт минут пятнадцать.

Момент, когда браузер впервые показывает **вашу** страницу, — важный. Не потому, что страница получится красивой (она получится страшной), а потому, что с этой секунды интернет перестаёт быть чем-то, что делают «где-то там другие люди».

Единственная просьба: **набирайте руками**. Не копируйте мышкой. Пальцы запоминают синтаксис быстрее, чем глаза.

РАЗБИРАЕМСЯ**Делаем страницу****► Шаг 1. Заводим папку**

Откройте терминал и выполните (или сделайте то же самое мышкой в проводнике):

```
cd ~  
mkdir -p projects/moi-magazin  
cd projects/moi-magazin
```

Мы в папке будущего магазина. Отсюда и начнём.

► Шаг 2. Открываем папку в редакторе

Запустите VS Code, выберите *Файл* → *Открыть папку* и укажите `projects/moi-magazin`.

Слева появится пустое дерево файлов. Это ваш проект.

Почему открывать именно папку, а не файл: редактор тогда видит весь проект целиком — может искать по всем файлам, подсказывать пути, показывать структуру. Привыкайте работать папками.

► Шаг 3. Создаём файл

В панели слева нажмите иконку «новый файл» и назовите его:

```
index.html
```

Имя важное, не придумывайте своё. `index.html` — это имя по умолчанию. Когда браузер приходит по адресу сайта, а конкретная страница не указана, сервер отдаёт именно `index.html`. Как «главная дверь» дома.

► Шаг 4. Пишем

Наберите это. Именно наберите:

```
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="UTF-8">
    <title>Моя первая страница</title>
  </head>
  <body>
    <h1>Привет! Это моя первая страница.</h1>
    <p>Я сделал её сам, и она работает.</p>
  </body>
</html>
```

Сохраните: Ctrl+S (на Mac Cmd+S).

► Шаг 5. Открываем в браузере

Найдите файл в проводнике и дважды щёлкните по нему. Откроется браузер.

Вот что вы увидите:

Привет! Это моя первая страница.

Я сделал её сам, и она работает.

Работает. Вы только что сделали веб-страницу.

Посмотрите на адресную строку — там будет что-то вроде

`file:///Users/firdavs/projects/moi-magazin/index.html`. Обратите внимание на `file://` — это значит «файл с моего компьютера», а не из интернета. Пока так и нужно.

РАЗБОР ПО СЛОВАМ

Разбор по словам

Теперь разберём каждую строчку. Ни одна не написана «просто потому что».

► Строка 1: `<!DOCTYPE html>`

```
<!DOCTYPE html>
```

Читается «доктайп эйч-ти-эм-эл». Это **не тег**, а объявление: «дорогой браузер, дальше идёт современный HTML, показывай по современным правилам».

Зачем нужно: если его не написать, браузер включит «режим совместимости» — притворится браузером из 1999 года. Вёрстка поедет, и вы будете долго искать причину.

Пишется **всегда первой строкой**, ровно так, один раз на страницу. Больше о нём думать не нужно.

► Строки 2: `<html lang="ru">`

```
<html lang="ru">
```

ЧАСТЬ

ЧТО ЭТО

```
<html>
```

Корневой элемент. Внутри него живёт вся страница. Открывается в начале, закрывается в самом конце

```
lang="ru"
```

Атрибут. Сообщает: содержимое на русском

Вот и новое понятие — **атрибут**.

Атрибут — это уточнение к тегу. Пишется внутри уголков, после имени тега, в формате `имя="значение"`.

Аналогия: тег — это существительное, атрибут — прилагательное. `<html>` — «страница», `lang="ru"` — «русскоязычная».

Зачем `lang` на практике: браузер поймёт, на каком языке текст, и правильно предложит перевод. Программы для незрячих прочитают текст с русским произношением, а не с английским. Поисковики покажут ваш сайт русскоязычным пользователям.

► Строки 3–6: `<head>` — служебная часть

```
<head>
  <meta charset="UTF-8">
  <title>Моя первая страница</title>
</head>
```

`<head>` («хед», голова) — это служебная информация о странице. Всё, что внутри `<head>`, на самой странице не видно.

Аналогия: это как поля на конверте — адрес, индекс, марка. Само письмо внутри, а это данные о письме.

СТРОКА	ЧТО ДЕЛАЕТ
<code><meta charset="UTF-8"></code>	Кодировка. Объясняет браузеру, как понимать буквы
<code><title></code>	Название страницы. Оно видно на вкладке браузера и в результатах поиска

Про `charset="UTF-8"` — запомните намертво.

Компьютер хранит не буквы, а числа. Кодировка — это таблица «какое число какой букве соответствует». UTF-8 — таблица, в которой есть всё: русский, английский, таджикский, китайский, эмодзи.

Если эту строчку не написать, браузер начнёт гадать. И русский текст превратится в `ЉŹŃЄĐ,Đ²ĐµŃ,` — знаменитые «кракозябры».

Правило: `<meta charset="UTF-8">` пишется всегда, в каждом файле, без исключений.

Заметьте ещё: у `<meta>` нет закрывающего тега. Такие теги называются **одиночными** — им нечего в себе содержать, они сами по себе законченная команда. Одиночных тегов немного: `<meta>`, `<img>`, `<br>`, `<input>`, `<hr>`.

► Строки 7–10: `<body>` — то, что видно

```
<body>
  <h1>Привет! Это моя первая страница.</h1>
  <p>Я сделал её сам, и она работает.</p>
</body>
```

`<body>` («боди», тело) — всё видимое содержимое страницы. Текст, картинки, кнопки, меню — всё живёт здесь.

Правило простое: не видно на экране → это в `<head>`. Видно → это в `<body>`.

`<h1>` и `<p>` вы уже встречали в первой главе: заголовок и абзац.

► Общая схема любой HTML-страницы

<code><!DOCTYPE html></code>	← «дальше современный HTML»
<code><html></code>	← вся страница
<code><head></code>	← служебное, невидимое
кодировка	
название вкладки	
<code></head></code>	
<code><body></code>	← видимое содержимое
заголовки	
текст	
картинки	
<code></body></code>	
<code></html></code>	

Эта структура **одинакова у всех страниц в интернете**. Откройте код любого сайта — увидите её же. Через неделю вы будете набирать её не задумываясь.

► Про отступы

Заметили, что строки внутри `<head>` сдвинуты вправо? Это **отступы**, и они делаются специально.

Браузеру они не нужны — ему всё равно. Отступы нужны **человеку**: по ним сразу видно, что внутри чего лежит.

Правило: каждый вложенный уровень сдвигается вправо (обычно на 4 пробела или Tab).

Код без отступов — как текст без абзацев и знаков препинания: прочитать можно, но мучительно.

РАЗБИРАЕМСЯ

Делаем страницу похожей на магазин

Первая страница работает. Теперь наполним её чем-то осмысленным.

Замените содержимое `<body>` на это:

```
<body>
  <h1>Автозапчасти Firdavs</h1>
  <p>Запчасти для японских и немецких автомобилей. Худжанд, доставка по Таджи

  <h2>Тормозные колодки Bosch</h2>
  <p>Артикул: 0986424815</p>
  <p>Цена: 250 сомони</p>
  <p>В наличии: 7 штук</p>
  <button>Купить</button>

  <h2>Масляный фильтр Mann</h2>
  <p>Артикул: W71280</p>
  <p>Цена: 45 сомони</p>
  <p>В наличии: 23 штуки</p>
  <button>Купить</button>
</body>
```

Не забудьте поменять и `<title>` — пусть будет «Автозапчасти — мой магазин».

Сохраните и обновите страницу в браузере клавишей F5.

Автозапчасти Firdavs

Запчасти для японских и немецких автомобилей. Худжанд, доставка по Таджикистану.

Тормозные колодки Bosch

Артикул: 0986424815

Цена: 250 сомони

В наличии: 7 штук

Купить

Масляный фильтр Mann

Артикул: W71280

Цена: 45 сомони

В наличии: 23 штуки

Купить

Вот честный результат — ровно то, что вы увидите у себя.

Давайте признаем прямо: **выглядит ужасно**. Чёрный текст на белом, всё в одну колонку, кнопки серые и системные. Ни один покупатель на таком сайте ничего не купит.

И это абсолютно правильный результат. Потому что мы написали только HTML — а HTML отвечает за содержание, не за красоту. Помните первую главу? Скелет.

Смотрите, что при этом уже **работает**:

- заголовки крупнее обычного текста, а `<h1>` крупнее `<h2>` ;
- абзацы отделены друг от друга воздухом;
- кнопки нажимаются — можете щёлкнуть, они «продавливаются»;
- на вкладке браузера видно название из `<title>` ;
- русский текст читается, кракозябр нет — спасибо `charset="UTF-8"` .

Всё это браузер сделал сам, по умолчанию, потому что вы **правильно назвали части**. Сказали «это заголовок» — он и оформлен как заголовок.

Красоту наведём в третьей части, начиная с главы 9. За один вечер эта же страница превратится в нормальный магазин. Но фундамент — вот он, уже стоит.

Открылся текст кода вместо страницы. Файл сохранился как `index.html.txt`. На Windows расширения по умолчанию скрыты. Лечение: Проводник → Вид → поставить галочку «Расширения имён файлов», потом переименовать.

Вместо русского — `ÐġÑ€Ð, Ð²ÐµÑ`. Забыли `<meta charset="UTF-8">` или сохранили файл не в UTF-8. VS Code по умолчанию сохраняет правильно, так что почти всегда дело в забытой строчке.

Поменял файл, а в браузере всё по-старому. Сначала проверьте, что файл сохранён (в VS Code у несохранённого файла точка на вкладке). Потом нажмите **Ctrl+F5** — обновление мимо кэша.

Забыл закрыть тег. Написали `<h1>` и не написали `</h1>` — дальше вся страница станет огромным заголовком. Браузер не ругается, он просто делает как понял. VS Code обычно закрывает теги сам, но проверять полезно.

Кавычки «не те». Атрибуты пишутся в прямых кавычках: `lang="ru"`. Если скопировать код из Word или мессенджера, кавычки могут превратиться в «ёлочки» — и атрибут сломается. Ещё один довод набирать руками.

Задача 4.1. Сделайте страницу из главы у себя. Убедитесь, что она открывается и русский текст читается.

Задача 4.2. Поменяйте `<title>` на своё имя. Сохраните, обновите браузер. Где именно изменился текст? (Подсказка: смотрите не на страницу.)

Задача 4.3. Добавьте в каталог третий товар — на ваш выбор, с названием, артикулом, ценой и наличием. Скопируйте структуру существующих.

Задача 4.4. Специально сломайте страницу и посмотрите, что будет:

1. Уберите `</h1>` у первого заголовка. Обновите. Что стало со страницей?
2. Верните обратно.
3. Уберите строку `<meta charset="UTF-8">`. Обновите. Что стало с русским текстом?
4. Верните обратно.

Ломать специально — лучший способ понять, зачем нужна каждая строчка. Так вы потом узнаете симптом мгновенно.

Задача 4.5. Попробуйте `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>` подряд с одинаковым текстом. Как меняется размер?

Задача 4.6. Откройте свою страницу и нажмите **F12** → **Elements**. Найдите там `<h1>`, который написали. Разверните дерево. Узнаёте свою структуру?

Задача 4.7. Откройте на любом крупном сайте код страницы (правая кнопка → «Посмотреть код страницы») и найдите в нём `<!DOCTYPE html>`, `<head>`, `<body>` и `<meta charset`. Убедитесь: у всех одно и то же.

Ответы — в Приложении Г.

В БОЮ

Откройте в этом репозитории файл `index.php` — главную страницу настоящего магазина.

Кода там много и он пока непонятен. Но найдите глазами `<head>`, `<meta charset`, `<title>` и `<body>`. Они там есть, ровно в том же порядке, что и в вашем файле.

Разница между вашей страницей и боевой — не в структуре. Структура одна. Разница в количестве наполнения. А его мы будем добавлять шестьдесят глав.

ИТОГ

- Страница называется `index.html` — это имя главной по умолчанию.
- `<!DOCTYPE html>` — первая строка всегда, включает современный режим.
- `<html lang="ru">` — вся страница, язык русский.
- `<head>` — служебное и невидимое: кодировка, название вкладки.
- `<body>` — всё, что видно человеку.
- `<meta charset="UTF-8">` обязателен, иначе русский превратится в кракозябры.
- Атрибут — уточнение к тегу: `имя="значение"`.
- Одиночные теги (`<meta>`, `<img>`, `<br>`) не закрываются.
- Отступы нужны человеку, а не браузеру. Делайте их всегда.
- Страница без CSS выглядит ужасно — и это нормально. HTML отвечает за смысл.

Со следующей главы начинается настоящий HTML: разберём теги по-серьёзному и научимся размечать что угодно.

Теги и структура документа

ЗАЧЕМ ЭТО

Зачем эта глава

В прошлой главе вы написали страницу и она заработала. Но написали вы её «по образцу» — как переписывают иероглифы, не зная языка.

Пора выучить язык. В этой главе разберёмся:

- что такое тег на самом деле и по каким правилам он живёт;
- почему одни элементы занимают всю строку, а другие — только своё место;
- зачем нужны `<div>` и `<span>`, если они ничего не делают;
- и главное — что такое **семантика** и почему из-за неё сайты попадают или не попадают в поиск.

После этой главы вы перестанете «списывать» и начнёте писать HTML осмысленно.

РАЗБИРАЕМСЯ

Что такое тег

Тег — это метка, которой вы называете кусок содержимого.

Вы не «рисуете заголовок». Вы говорите браузеру: вот это — заголовок. А как его показать, браузер решит сам.

```
<h1>Автозапчасти Firdavs</h1>
```

Здесь три части:

ЧАСТЬ	НАЗВАНИЕ	ЧТО ДЕЛАЕТ
<code><h1></code>	открывающий тег	«Внимание, начинается заголовок»
Автозапчасти Firdavs	содержимое	То, что видит человек
<code></h1></code>	закрывающий тег	«Заголовок закончился»

Всё вместе — элемент.

► Правило вложенности

Элементы можно класть друг в друга. Но закрывать нужно в обратном порядке — как матрёшки.

Правильно:

```
<p>Цена: <strong>250 сомони</strong></p>
```

Открыли `<p>`, внутри открыли `<strong>`, закрыли `</strong>`, потом закрыли `</p>`.
Внутренняя матрёшка закрылась первой.

Неправильно:

```
<p>Цена: <strong>250 сомони</p></strong>
```

Тут `<p>` закрылся раньше, чем `<strong>`. Браузер не выдаст ошибку — он попытается догадаться. И часто догадается неправильно, а вы полчаса будете искать, почему поехала вёрстка.

Запомните так: что открыли последним — закрываете первым.

► Атрибуты — уточнения к тегу

Атрибут добавляет тегу подробностей. Пишется внутри открывающего тега:

```
<html lang="ru">

<a href="/catalog">Каталог</a>
```

Формула всегда одна:

имя="значение"

ПРАВИЛО

ПОЯСНЕНИЕ

Кавычки **прямые**

`lang="ru"`, а не `lang=«ru»`

Атрибутов может быть много

Разделяются пробелом

Порядок не важен

`` = ``

Некоторые атрибуты без значения

`<input required>` — просто «обязательное»

Два атрибута, которые есть почти у любого тега и понадобятся в главе 9:

- **class** — «этот элемент относится к такой-то группе». По классу CSS будет находить элементы и красить: `<div class="tovar">`.
- **id** — «у этого элемента уникальное имя». На странице **id** должен быть один: `<div id="korzina">`.

Разница простая: **класс** — как «студент группы 101» (таких много). **id** — как номер паспорта (такой один).

РАЗБИРАЕМСЯ

Блочные и строчные элементы

Вот важное различие, которое объясняет половину «непонятного поведения» вёрстки.

► Блочные — занимают всю строку

Блочный элемент **всегда** начинается с новой строки и растягивается на всю ширину, даже если содержимого в нём одно слово.

Примеры: `<h1>` ... `<h6>`, `<p>`, `<div>`, `<ul>`, `<table>`, `<form>`.

Аналогия: **абзац в книге**. Даже если в нём одно слово, он всё равно занимает отдельную строку и никого рядом не пускает.

► Строчные — занимают только своё место

Строчный элемент **встраивается в строку** и занимает ровно столько, сколько нужно его содержимому.

Примеры: `<a>`, `<strong>`, `<em>`, `<span>`, `<img>`.

Аналогия: выделенное слово в предложении. Оно внутри строки, не разрывает её.

► Сравните на примере

```
<p>Первый абзац</p>
<p>Второй абзац</p>
```

Встанут друг под другом, потому что `<p>` — блочный.

```
<strong>Первое</strong>
<strong>Второе</strong>
```

Встанут в одну строку, потому что `<strong>` — строчный.

Практический вывод. Когда вы недоумеваете «почему эти два блока не встают рядом» или «почему кнопка растянулась на весь экран» — почти всегда ответ здесь. В главе 11 мы научимся управлять этим через CSS.

РАЗБИРАЕМСЯ

и — коробки без смысла

Два тега, которые вы будете использовать чаще всех.

ТЕГ	ТИП	СМЫСЛ	ЗАЧЕМ
<code><div></code>	блочный	никакого	Сгруппировать блок содержимого
<code></code>	строчный	никакого	Выделить кусочек внутри строки

Оба ничего не означают и никак не выглядят. Это чистые контейнеры.

► Зачем нужна пустая коробка

Возьмём карточку товара из прошлой главы:


```
<h2>Тормозные колодки Bosch</h2>
<p>Артикул: 0986424815</p>
<p>Цена: 250 сомони</p>
<button>Купить</button>
```

Четыре отдельных элемента. Браузер не знает, что они связаны — для него это просто заголовок, два абзаца и кнопка, идущие подряд.

Заворачиваем в коробку:

```
<div class="tovar">
  <h2>Тормозные колодки Bosch</h2>
  <p>Артикул: 0986424815</p>
  <p>Цена: 250 сомони</p>
  <button>Купить</button>
</div>
```

Теперь это одна вещь, у которой есть имя — `товар`. И можно сказать одной строчкой CSS: «все элементы с классом `товар` сделать белыми, с рамкой, скруглёнными углами и отступом». Все карточки оформятся разом.

► А `<span>` зачем

Когда нужно выделить кусочек **внутри** строки:

```
<p>Цена: <span class="cena">250</span> сомони</p>
```

Теперь можно покрасить только число, не трогая слова вокруг.

Правило выбора: нужно завернуть блок содержимого — `<div>`. Нужно выделить пару слов внутри текста — `<span>`.

РАЗБИРАЕМСЯ

Семантика: главная идея HTML

А вот теперь — самое важное в этой главе.

► Проблема

Технически можно построить весь сайт из `<div>`. Вообще весь:

```
<div class="zagolovok">Автозапчасти</div>
<div class="menu">Каталог Доставка Контакты</div>
<div class="tovar">Колодки Bosch</div>
<div class="podval">© 2026</div>
```

Работать будет. Выглядеть после CSS — прилично. Но это плохой код, и вот почему.

Для браузера, поисковика и программы для незрячих это четыре одинаковые безымянные коробки. Слово `zagolovok` в атрибуте `class` ничего им не говорит — они читают теги, а не ваши выдумки. С тем же успехом там могло быть написано `korobka1`.

► Решение: теги, у которых есть смысл

В HTML есть **семантические теги** — теги со значением. Вместо безымянной коробки вы говорите, что это за часть страницы:

ТЕГ	ЧТО ОЗНАЧАЕТ	ЧТО БЫЛО БЫ ВМЕСТО
<code><header></code>	Шапка: логотип, название	<code><div class="header"></code>
<code><nav></code>	Меню, навигация	<code><div class="menu"></code>
<code><main></code>	Основное содержимое страницы	<code><div class="content"></code>
<code><article></code>	Самостоятельный кусок: товар, статья	<code><div class="item"></code>
<code><section></code>	Раздел страницы	<code><div class="block"></code>
<code><aside></code>	Боковая колонка, дополнительное	<code><div class="sidebar"></code>
<code><footer></code>	Подвал: контакты, копирайт	<code><div class="footer"></code>

Пишутся они **точно так же**, как `<div>`, и ведут себя так же — блочные, без вида. Разница только в одном: у них **есть смысл**.

► Как выглядит правильная страница

```
<body>
  <header>
    <h1>Автозапчасти Firdavs</h1>
  </header>

  <nav>
    <a href="/catalog">Каталог</a>
    <a href="/delivery">Доставка</a>
    <a href="/contacts">Контакты</a>
  </nav>

  <main>
    <article>
      <h2>Тормозные колодки Bosch</h2>
      <p>Цена: 250 сомони</p>
      <button>Купить</button>
    </article>

    <article>
      <h2>Масляный фильтр Mann</h2>
      <p>Цена: 45 сомони</p>
      <button>Купить</button>
    </article>
  </main>

  <footer>
    <p>© 2026 Автозапчасти Firdavs. Худжанд.</p>
  </footer>
</body>
```

Автозапчасти Firdavs

[Каталог](#) [Доставка](#) [Контакты](#)

Тормозные колодки Bosch

Цена: 250 сомони

Купить

Масляный фильтр Mann

Цена: 45 сомони

Купить

© 2026 Автозапчасти Firdavs. Худжанд.

Выглядит так же скучно, как раньше, — семантика не про внешность. Но теперь структура страницы **читается** по одному только коду.

► Что вы за это получаете

Три очень практичных выигрыша:

1. **Поисковики понимают страницу.** Google видит `<main>` и знает: вот здесь главное, а `<nav>` и `<footer>` можно не учитывать. Правильная семантика — часть SEO, то есть попадания сайта в поиск.

2. **Незрячие люди могут пользоваться сайтом.** Программы-чтецы умеют перепрыгивать по разделам: «перейти к основному содержимому», «перейти к меню». Если у вас одни `<div>`, прыгать не по чему — человек вынужден слушать всё подряд, включая меню на каждой странице.

3. **Код читаемый.** Через полгода вы откроете свой файл. `</div></div></div></div>` в конце — ад. `</main></body>` — понятно.

Правило на всю жизнь: *есть подходящий по смыслу тег — берите его.* `<div>` — когда подходящего нет.

Новые слова этой главы, собранные вместе:

СЛОВО	ЧТО ОЗНАЧАЕТ
Тег	Метка в уголках: <code><p></code>
Элемент	Открывающий тег + содержимое + закрывающий
Атрибут	Уточнение внутри тега: <code>class="tovar"</code>
Вложенность	Элементы внутри элементов, как матрёшки
Блочный	Занимает всю строку: <code><div></code> , <code><p></code> , <code><h1></code>
Строчный	Занимает своё место в строке: <code></code> , <code><a></code> , <code></code>
Семантика	Смысл тега. <code><header></code> вместо <code><div class="header"></code>
<code>class</code>	Имя группы. Таких может быть много
<code>id</code>	Уникальное имя. Такой один на странице

Ставить `<h1>` несколько раз. `<h1>` — главный заголовок страницы, он один. Дальше по убыванию: `<h2>` для разделов, `<h3>` внутри них. Не прыгайте через уровень (`<h1>` → `<h3>`) и не выбирайте заголовок по размеру — размер настроим в CSS.

Выбирать тег по внешнему виду. «Мне нужен мелкий текст, возьму `<h6>` » — так нельзя. Тег выбирают по смыслу, внешний вид задаёт CSS. Это ошибка, которую совершают почти все новички.

Заворачивать всё подряд в лишние `<div>` . Если внутри коробки лежит ровно один элемент и коробка ничего не группирует — она не нужна. Лишние обёртки делают код нечитаемым.

Пробел внутри тега. `< p>` — не работает. Между `<` и именем тега пробела быть не должно.

Русские буквы в `class` и `id` . Технически возможно, практически — источник проблем. Только латиница: `class="tovar"` , а не `class="товар"` .

Задача 5.1. Перепишите свою страницу из главы 4, добавив семантические теги:

`<header>` с названием магазина, `<nav>` с тремя ссылками, `<main>` с товарами (каждый в `<article>`), `<footer>` с копирайтом.

Задача 5.2. Для каждого определите — блочный или строчный: `<p>`, `<span>`, `<div>`, `<a>`, `<h2>`, `<strong>`, `<article>`, `<em>`.

Задача 5.3. Найдите ошибку и объясните, в чём она:

```
<p>Цена: <strong>250 сомони</p></strong>
```

Задача 5.4. Замените `<div>` на подходящий семантический тег:

```
<div class="header"> ... </div>
<div class="menu"> ... </div>
<div class="content"> ... </div>
<div class="sidebar"> ... </div>
<div class="footer"> ... </div>
```

Задача 5.5. В каком случае берут `<div>`, а в каком `<span>`? Приведите свой пример на каждый.

Задача 5.6. Откройте крупный интернет-магазин, нажмите F12 → Elements и поищите теги `<header>`, `<nav>`, `<main>`, `<footer>`. Есть ли они? У некоторых крупных сайтов их нет — сплошные `<div>`. Как думаете, почему так вышло?

Задача 5.7. Объясните своими словами, почему `<div class="zagolovok">` хуже, чем `<h1>`. Два предложения.

Ответы — в Приложении Г.

Откройте `index.php` в корне репозитория и поищите в нём `<div class=`. Найдёте много.

Это правда жизни: боевой код никогда не бывает идеальным. Сайт autodoc.tj построен на готовом шаблоне Mazlay, а шаблоны обычно щедрЫ на `<div>`.

Важный вывод, который стоит усвоить сразу: **работающий код с недостатками лучше, чем идеальный код, которого нет**. Стремитесь к семантике в том, что пишете сами, но не переписывайте рабочий сайт ради красоты — у бизнеса есть задачи поважнее.

ИТОГ

Итог

- **Элемент** = открывающий тег + содержимое + закрывающий.
- Вложенность закрывается **в обратном порядке**, как матрёшки.
- **Атрибут** — уточнение: `имя="значение"`, кавычки прямые.
- `class` — имя группы (много), `id` — уникальное имя (один).
- **Блочные** занимают всю строку, **строчные** — только своё место.
- `<div>` — блочная коробка, `<span>` — строчная. Обе без смысла.
- **Семантика**: `<header>`, `<nav>`, `<main>`, `<article>`, `<footer>` — то же самое, но со смыслом. Нужны для поиска, для незрячих и для читаемости.
- Тег выбирают **по смыслу**, а не по внешнему виду.

В следующей главе наполним каркас содержимым: текст, ссылки, картинки и списки.

Текст, ссылки, картинки, списки

ЗАЧЕМ ЭТО

Зачем эта глава

Каркас страницы у вас есть. Теперь научимся наполнять его тем, из чего состоит любой сайт: текстом, ссылками, картинками и списками.

Тегов здесь будет много, но пугаться не надо — это как выучить названия инструментов в мастерской. Каждый простой, каждый делает одну вещь. К концу главы вы сможете разметить любую страницу с текстом.

РАЗБИРАЕМСЯ

Текст

► Заголовки: от `<h1>` до `<h6>`

```
<h1>Автозапчасти Firdavs</h1>
<h2>Тормозная система</h2>
<h3>Колодки</h3>
```

Шесть уровней. Работают как оглавление книги: часть → глава → раздел.

Три правила, которые нарушают все:

1. `<h1>` на странице один. Это её название. Не два, не три.
2. Не прыгать через уровень. После `<h1>` идёт `<h2>`, а не `<h3>`.
3. Не выбирать по размеру. Нужен мелкий заголовок — берите правильный уровень и уменьшайте его в CSS.

Почему это важно: по заголовкам строится «оглавление» страницы. Им пользуются поисковики и программы для незрячих. Сломанная иерархия — сломанное оглавление.

► Абзацы и переносы

```
<p>Первый абзац текста.</p>
<p>Второй абзац текста.</p>
```

Абзац — `<p>`, блочный, сам отделяется отступами.

Важная особенность HTML: он не видит ваших переносов строк и лишних пробелов.

```
<p>Слово          другое
слово</p>
```

Браузер покажет: `Слово другое слово`. Все пробелы подряд он сожмёт в один, а перенос строки посчитает пробелом.

Это не ошибка, а правило: пустое место в коде нужно человеку, а не браузеру. Поэтому можно спокойно делать отступы и переносы для читаемости — на результат не влияет.

Нужен настоящий перенос строки внутри абзаца — есть одиночный тег `<br>`:

```
<p>Худжанд, ул. Ленина, 15<br>Телефон: +992 92 777 77 77</p>
```

⚠ Не используйте `<br><br>` для отступов между абзацами. Для этого есть `<p>` и отступы в CSS. `<br>` — только для случаев, когда перенос часть содержания: адрес, стихи, реквизиты.

► Выделение: `<strong>` и `<em>`

```
<p>Цена: <strong>250 сомони</strong></p>
<p>Товар <em>под заказ</em>, срок 5 дней.</p>
```

ТЕГ	КАК ВЫГЛЯДИТ	ЧТО ОЗНАЧАЕТ
<code></code>	жирный	Важно. Обратите внимание
<code></code>	курсив	Смысловое ударение

Есть ещё `<b>` (жирный) и `<i>` (курсив). Выглядят так же, но не несут смысла — просто «сделай жирным». Программа для незрячих выделит голосом `<strong>` и проигнорирует `<b>`.

Правило: важное — `<strong>`, ударение — `<em>`, а `<b>` и `<i>` не трогайте.

► Остальные полезные теги

```
<p>Артикул: <code>0986424815</code></p>
<p>Цена: <s>300</s> 250 сомони</p>
<p>Гарантия <small>(при сохранении чека)</small></p>
<blockquote>Отличные колодки, тормозит как новая.</blockquote>
<hr>
```

ТЕГ	ЧТО ДЕЛАЕТ	ГДЕ ПРИГОДИТСЯ
<code><code></code>	Моноширинный шрифт	Артикулы, коды, куски кода
<code><s></code>	Зачёркнутый	Старая цена при скидке
<code><small></code>	Мелкий текст	Примечания, сноски
<code><blockquote></code>	Цитата	Отзывы покупателей
<code><hr></code>	Линия-разделитель	Разделить разделы

РАЗБИРАЕМСЯ

Ссылки

Ссылка — то, ради чего интернет вообще существует. Тег `<a>`, от английского *anchor*, якорь.

```
<a href="/catalog">Каталог</a>
```

ЧАСТЬ	ЧТО ЭТО
<code><a></code>	Тег ссылки. Строчный — встраивается в текст
<code>href</code>	Куда ведём. От <i>hypertext reference</i>
Каталог	Что видит и нажимает человек

Без `href` ссылка не работает. Это её единственный обязательный атрибут.

► Виды адресов

Здесь новички путаются чаще всего, поэтому разберём подробно.

```
<a href="https://autodoc.tj/catalog">На другой сайт</a>
<a href="/catalog">От корня сайта</a>
<a href="catalog.html">Рядом с текущим файлом</a>
<a href="../index.html">На папку выше</a>
<a href="#dostavka">К месту на этой же странице</a>
```

ЗАПИСЬ	НАЗВАНИЕ	КУДА ВЕДЁТ
<code>https:// ...</code>	Абсолютный	Полный адрес. На чужие сайты только так
<code>/catalog</code>	От корня	От начала вашего сайта. Работает с любой страницы
<code>catalog.html</code>	Относительный	Файл в той же папке
<code>../index.html</code>	Относительный	<code>..</code> — подняться на папку выше
<code>#dostavka</code>	Якорь	К элементу с <code>id="dostavka"</code> на этой странице

Практический совет: внутри своего сайта используйте адреса **от корня** — `/catalog`. Относительные ломаются, когда вы переносите файл в другую папку, и это одна из самых раздражающих ошибок.

► Особые ссылки

```
<a href="tel:+992927777777">+992 92 777 77 77</a>
<a href="mailto:info@autodoc.tj">Написать нам</a>
```

На телефоне первая ссылка сразу звонит. Вторая открывает почтовую программу. Мелочь, а покупателю приятно.

► Открыть в новой вкладке

```
<a href="https://bosch.com" target="_blank" rel="noopener">Сайт Bosch</a>
```

`target="_blank"` — открыть в новой вкладке. `rel="noopener"` — обязательная добавка для безопасности: без неё чужая страница получает доступ к вашей.

Когда открывать в новой вкладке: ссылки на чужие сайты. Свои страницы — в той же вкладке, иначе завалите человеку браузер.

РАЗБИРАЕМСЯ

Картинки

```

```

`<img>` — одиночный тег, закрывать не нужно. Два обязательных атрибута:

АТТРИБУТ	ЧТО ЗНАЧИТ
<code>src</code>	Откуда брать. Путь к файлу (<i>source</i> — источник)
<code>alt</code>	Что здесь изображено словами

► Почему `alt` обязателен

Про него всегда забывают, а он делает три вещи:

1. Показывается, если картинка не загрузилась. Битый путь, медленный интернет — человек увидит хотя бы описание.
2. Его читают программы для незрячих. Без `alt` человек услышит «изображение» — и всё.
3. По нему картинки находят в поиске. Поиск по картинкам работает именно на `alt`.

Как писать `alt` :

```
 ✓  
 ✗  
 ⚠
```

Пустой `alt=""` — это осознанное «картинка декоративная, читать нечего». Для узоров и разделителей так правильно. Для товара — нет.

► Размеры

```

```

Указывать размеры полезно: браузер заранее резервирует место, и страница не «прыгает» во время загрузки. Знакомое раздражение, когда читаешь текст, а он уезжает вниз, — это как раз незадаанные размеры.

► Форматы

ФОРМАТ	ДЛЯ ЧЕГО
JPG	Фотографии товаров
PNG	Логотипы, картинки с прозрачностью
SVG	Иконки, схемы. Не мылятся при увеличении
WebP	Современный, весит меньше при том же качестве

⚠ Главная ошибка новичка с картинками: залить фото прямо с телефона. Такое фото весит 5 МБ. Двадцать товаров — 100 МБ на одной странице. С мобильного интернета это не откроется никогда. Фото для сайта сжимают до 100–300 КБ.

РАЗБИРАЕМСЯ

Списки

► Маркированный — когда порядок не важен

```
<ul>
  <li>Тормозные колодки</li>
  <li>Тормозные диски</li>
  <li>Тормозная жидкость</li>
</ul>
```

`<ul>` — *unordered list*, нумерованный список. `<li>` — *list item*, пункт.

► Нумерованный — когда порядок важен

```
<ol>
  <li>Выберите товар</li>
  <li>Добавьте в корзину</li>
  <li>Оформите заказ</li>
</ol>
```

`<ol>` — *ordered list*. Номера браузер расставляет сам — не пишите их руками, иначе при вставке нового пункта придётся перенумеровывать всё.

Правило: можно переставить пункты местами без потери смысла — `<ul>`. Нельзя (это шаги, рейтинг, инструкция) — `<ol>`.

► Вложенные списки

```
<ul>
  <li>Тормозная система
    <ul>
      <li>Колодки</li>
      <li>Диски</li>
    </ul>
  </li>
  <li>Двигатель</li>
</ul>
```

Внимание: вложенный список кладут **внутри** `<li>`, а не между ними. Так делают многоуровневые меню каталога.

► Список описаний

Редкий, но очень удобный для характеристик товара:

```
<dl>
  <dt>Производитель</dt>
  <dd>Bosch</dd>
  <dt>Артикул</dt>
  <dd>0986424815</dd>
  <dt>Гарантия</dt>
  <dd>12 месяцев</dd>
</dl>
```

`<dl>` — список, `<dt>` — термин, `<dd>` — описание.

Страница товара, использующая всё из этой главы:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Тормозные колодки Bosch — Автозапчасти Firdavs</title>
</head>
<body>
  <header>
    <h1>Автозапчасти Firdavs</h1>
  </header>

  <nav>
    <a href="/">Главная</a> ·
    <a href="/catalog">Каталог</a> ·
    <a href="/contacts">Контакты</a>
  </nav>

  <main>
    <article>
      <h2>Тормозные колодки Bosch</h2>

      

      <p>Артикул: <code>0986424815</code></p>
      <p>Цена: <s>300</s> <strong>250 сомони</strong></p>
      <p>Состояние: <em>в наличии, 7 комплектов</em></p>

      <h3>Характеристики</h3>
      <dl>
        <dt>Производитель</dt>
        <dd>Bosch, Германия</dd>
        <dt>Подходит для</dt>
        <dd>Toyota Camry 2011-2017</dd>
        <dt>Гарантия</dt>
        <dd>12 месяцев</dd>
      </dl>
    </article>
  </main>
</body>
</html>
```



```

<h3>Что входит в комплект</h3>
<ul>
  <li>Колодки передние – 4 шт.</li>
  <li>Смазка направляющих</li>
  <li>Датчик износа</li>
</ul>

<h3>Как заказать</h3>
<ol>
  <li>Позвоните или напишите нам</li>
  <li>Уточните марку и год автомобиля</li>
  <li>Заберите в Худжанде или закажите доставку</li>
</ol>

<button>Купить</button>

<hr>

<blockquote>Поставил на Камри – тормозит мягко, скрипа нет. Рекомендую!</blockquote>
<p><small>Отзыв покупателя, март 2026</small></p>
</article>
</main>

<footer>
  <p>Худжанд, ул. Ленина 15<br>
  Телефон: <a href="tel:+992927777777">+992 92 777 77 77</a></p>
  <p><small>© 2026 Автозапчасти Firdavs</small></p>
</footer>
</body>
</html>

```

Автозапчасти Firdavs

[Главная](#) · [Каталог](#) · [Контакты](#)

Тормозные колодки Bosch

Артикул: 0986424815

Цена: ~~300~~ **250 сомони**

Состояние: *в наличии, 7 комплектов*

Характеристики

Производитель

Bosch, Германия

Подходит для

Toyota Camry 2011–2017

Гарантия

12 месяцев

Что входит в комплект

- Колодки передние — 4 шт.
- Смазка направляющих
- Датчик износа

Как заказать

1. Позвоните или напишите нам
2. Уточните марку и год автомобиля
3. Заберите в Худжанде или закажите доставку

Купить

Поставил на Камри — тормозит мягко, скрипа нет. Рекомендую.

Отзыв покупателя, март 2026

Худжанд, ул. Ленина 15

Телефон: [+992 92 777 77 77](tel:+992927777777)

© 2026 Автозапчасти Firdavs

Опять некрасиво — и опять правильно. Зато посмотрите, сколько всего уже работает само по себе:

- заголовки выстроились по важности;
- ссылки синие и подчёркнутые, нажимаются;
- список с точками, нумерованный — с цифрами;
- зачёркнутая старая цена рядом с новой;
- цитата отодвинута от края;
- характеристики выстроились в два уровня;
- телефон на мобильном будет звонить одним касанием.

Всё это браузер сделал сам, потому что вы **правильно назвали каждую часть**. Добавим CSS — и та же страница станет магазином.

ГРАБЛИ

Грабли

Картинка не отображается. Три причины по частоте: неверный путь в `src` (проверьте регистр букв — `Kolodki.JPG` и `kolodki.jpg` для сервера разные файлы), файл не туда положили, опечатка в расширении.

`<br>` вместо абзацев. Признак кода, написанного наугад. Абзац — `<p>` .

Ссылка без `href` . `<a>Каталог</a>` — не ссылка, а просто текст.

Ссылки на весь сайт в новой вкладке. Раздражает. `target="_blank"` — только для чужих сайтов.

Фото по 5 МБ. Сожмите до 100–300 КБ, иначе сайт будет открываться минуту.

`alt="фото"` . Бессмысленно. Пишите, что на картинке, или ставьте пустой `alt=""` , если она декоративная.

ЗАДАЧИ

Задачи

Задача 6.1. Сделайте страницу товара из примера у себя. Возьмите свой товар, не колодки.

Задача 6.2. Найдите ошибки:

```
<a>Каталог</a>

<ul>
  <p>Первый пункт</p>
  <p>Второй пункт</p>
</ul>
<h1>Заголовок</h1>
<h1>Другой заголовок</h1>
```

Задача 6.3. Что покажет браузер? Ответьте, не запуская:

```
<p>Один      два
три</p>
```

Задача 6.4. Сделайте меню каталога вложенным списком, три раздела по два подраздела в каждом.

Задача 6.5. Разметьте свою визитку: имя `<h1>`, должность абзацем, телефон ссылкой `tel:`, почта ссылкой `mailto:`, три навыка списком.

Задача 6.6. Когда `<ul>`, а когда `<ol>` ?

1. Список ингредиентов плова.
2. Шаги приготовления плова.
3. Топ-5 самых продаваемых товаров.
4. Что входит в комплект поставки.

Задача 6.7. Скачайте любое фото, положите рядом с `index.html` и вставьте на страницу с осмысленным `alt`. Потом специально сломайте путь в `src` и обновите — увидите, как выглядит `alt` в деле.

Ответы — в Приложении Г.

Откройте `pages/` в этом репозитории и посмотрите, как в боевом коде выводятся характеристики товара и меню каталога. Найдёте те же `<ul>`, `<li>`, `<a>` и `<img>` — только значения в них подставляет РНР, а не человек руками.

Вот к чему мы идём: **разметку пишете вы один раз, а данные в неё подставляются из базы.** Одна страница товара обслуживает десять тысяч товаров.

ИТОГ

Итог

- Заголовки `<h1>` — `<h6>` — иерархия, а не размер. `<h1>` один на страницу.
- HTML **сжимает пробелы и переносы**. Форматируйте код как удобно.
- `<strong>` — важное, `<em>` — ударение. `<b>` и `<i>` не используйте.
- Ссылка `<a href="...">`. Внутри сайта — адреса **от корня**: `/catalog`.
- `tel:` и `mailto:` — звонок и письмо одним касанием.
- `` — **одиночный** тег. `alt` обязателен.
- Фото сжимать до 100–300 КБ. Не заливать прямо с телефона.
- `<ul>` — порядок не важен, `<ol>` — важен, `<dl>` — характеристики.

Дальше — таблицы и формы: научимся принимать данные от покупателя.

Таблицы и формы

ЗАЧЕМ ЭТО

Зачем эта глава

До сих пор мы только **показывали** информацию. Теперь научимся двум вещам посерьёзнее:

- **таблицы** — показывать данные, у которых есть строки и столбцы;
- **формы** — принимать данные от покупателя.

Форма — это первый раз, когда информация побежит в обратную сторону: не от сайта к человеку, а от человека к сайту. Регистрация, вход, поиск, заказ, отзыв — всё это формы.

Отправлять данные по-настоящему мы научимся в главе 24, когда появится РНР. Сейчас построим саму форму — и разберём каждое поле.

РАЗБИРАЕМСЯ

Таблицы

► Когда таблица уместна, а когда нет

Сразу главное правило, потому что его нарушают постоянно:

Таблица — для данных, у которых есть строки и столбцы. Прайс-лист, характеристики, расписание, отчёт о продажах.

Таблица — не для раскладки страницы. Двадцать лет назад сайты верстали таблицами, потому что другого способа не было. Сегодня для раскладки есть Flexbox и Grid (глава 11), а таблица в этой роли ломает мобильную версию и мешает незрячим.

► Из чего состоит таблица

```
<table>
  <thead>
    <tr>
      <th>Товар</th>
      <th>Артикул</th>
      <th>Цена</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Колодки Bosch</td>
      <td>0986424815</td>
      <td>250 сомони</td>
    </tr>
    <tr>
      <td>Фильтр Mann</td>
      <td>W71280</td>
      <td>45 сомони</td>
    </tr>
  </tbody>
</table>
```

ТЕГ	РАСШИФРОВКА	ЧТО ЭТО
<code><table></code>		Вся таблица
<code><thead></code>	<i>table head</i>	Шапка таблицы
<code><tbody></code>	<i>table body</i>	Тело: собственно данные
<code><tr></code>	<i>table row</i>	Строка
<code><th></code>	<i>table header</i>	Ячейка-заголовок. Жирная и по центру
<code><td></code>	<i>table data</i>	Обычная ячейка с данными

Логика такая: таблица состоит из **строк** (`<tr>`), а строка — из **ячеек** (`<th>` или `<td>`).

⚠ Частая путаница: люди пытаются описать таблицу по столбцам. Нельзя. HTML знает только строки. Столбцы получаются сами, когда в каждой строке одинаковое число ячеек.

► Объединение ячеек

```
<tr>
  <td colspan="2">Занимает два столбца</td>
  <td>Обычная</td>
</tr>
<tr>
  <td rowspan="2">Занимает две строки</td>
  <td>Ячейка</td>
</tr>
```

`colspan` — объединить по горизонтали, `rowspan` — по вертикали. Как в Excel.

► Подпись

```
<table>
  <caption>Прайс-лист на тормозную систему, март 2026</caption>
  ...
</table>
```

`<caption>` — заголовок таблицы. Пишется сразу после `<table>`. Полезен и людям, и незрячим.

РАЗБИРАЕМСЯ

Формы

Теперь главное в этой главе.

► Что такое форма

Форма — это конверт, в который складывают данные и отправляют на сервер.

```
<form action="/order.php" method="POST">
  <input type="text" name="phone">
  <button type="submit">Отправить</button>
</form>
```


ЧАСТЬ	ЧТО ДЕЛАЕТ
<code><form></code>	Сам конверт. Всё внутри отправится вместе
<code>action</code>	Куда отправить. Адрес обработчика на сервере
<code>method</code>	Как отправить. <code>GET</code> или <code>POST</code>
<code><input></code>	Поле для ввода
<code><button type="submit"></code>	Кнопка «отправить»

► GET или POST

Помните из главы 2? Теперь применим.

	GET	POST
Где едут данные	В адресной строке	Внутри письма, не видно
Видно в адресе	<code>?q=kolodki&brand=bosch</code>	Ничего не видно
Можно ли добавить в закладки	Да	Нет
Ограничение по размеру	Есть, около 2000 символов	Практически нет
Для чего	Поиск, фильтры	Заказы, вход, регистрация

Простое правило: данные, которые не жалко показать в адресной строке и которыми хочется поделиться ссылкой, — GET. Всё остальное — POST.

Пароль в адресной строке — это пароль в истории браузера, в логах сервера и в закладках. Поэтому вход — только POST.

► Атрибут `name` — самый важный

```
<input type="text" name="phone">
```

Без `name` поле не отправится вообще. Совсем. Человек заполнит, нажмёт кнопку, данные потеряются.

Почему: `name` — это **подпись на данных**. Сервер получит `phone=99292777777` и по слову `phone` поймёт, что это телефон. Без подписи он получит непонятную строку без имени и просто её выбросит.

Запомните: **нет** `name` — **нет данных**. Это ошибка номер один в формах.

РАЗБИРАЕМСЯ

Виды полей

`<input>` — один тег, но его поведение полностью меняется атрибутом `type`.

► Текстовые

```
<input type="text"      name="name"      placeholder="Ваше имя">
<input type="email"     name="email"     placeholder="pochta@example.com">
<input type="tel"       name="phone"     placeholder="+992 __ __ __ __">
<input type="password"  name="password">
<input type="number"    name="qty"       min="1" max="99" value="1">
<input type="search"    name="q"         placeholder="Поиск по каталогу">
```

TYPE

ЧТО ДАЁТ

<code>text</code>	Обычная строка
<code>email</code>	Браузер проверит, что есть <code>@</code> . На телефоне — клавиатура с <code>@</code>
<code>tel</code>	На телефоне откроется цифровая клавиатура
<code>password</code>	Символы прячутся за точками
<code>number</code>	Только числа, появляются стрелочки. <code>min</code> и <code>max</code> ограничивают
<code>search</code>	Поле поиска, у него появляется крестик очистки

Про мобильную клавиатуру — недооценённая мелочь. Поставили `type="tel"` — человек на телефоне сразу видит цифры вместо букв. Не поставили — он мучается, переключая раскладку. Такие детали и отличают удобный сайт от неудобного.

► Выбор

←!— Один из нескольких →

```
<input type="radio" name="dostavka" value="samovyvoz" id="d1">
<label for="d1">Самовывоз</label>
```

```
<input type="radio" name="dostavka" value="kurier" id="d2">
<label for="d2">Курьером</label>
```

←!— Галочка →

```
<input type="checkbox" name="soglasie" id="s1">
<label for="s1">Согласен на обработку данных</label>
```

Ключевая разница:

- **radio** — выбрать **одно** из нескольких. Чтобы они работали как группа, у них должен быть одинаковый `name`. Это самая частая ошибка: разные `name` — и можно выбрать все варианты сразу.
- **checkbox** — включить или выключить. Каждый сам по себе.

► Выпадающий список

```
<select name="brand">
  <option value="">Все бренды</option>
  <option value="bosch">Bosch</option>
  <option value="mann" selected>Mann</option>
  <option value="denso">Denso</option>
</select>
```

`value` — что уйдёт на сервер, текст между тегами — что видит человек. `selected` — вариант по умолчанию.

► Большое поле для текста

```
<textarea name="comment" rows="4" placeholder="Комментарий к заказу"></textarea>
```

⚠ У `<textarea>` есть закрывающий тег, в отличие от `<input>`. И значение по умолчанию пишется между тегами, а не в атрибут `value`.

► Прочее

```
<input type="date" name="dostavka_data">
<input type="file" name="foto">
<input type="hidden" name="tovar_id" value="42">
```

hidden — скрытое поле. Человек его не видит, но оно отправится вместе с формой. Так передают служебные данные: номер товара, защитный токен.

РАЗБИРАЕМСЯ

— обязательная вещь

```
<label for="phone">Телефон</label>
<input type="tel" id="phone" name="phone">
```

Связка простая: **for** у метки совпадает с **id** у поля.

Зачем это нужно:

1. Клик по надписи ставит курсор в поле. Особенно важно для маленьких галочек: попасть пальцем в квадратик 15×15 на телефоне трудно, а в надпись рядом — легко.
2. Программы для незрячих прочитают, что это за поле. Без метки человек услышит «поле ввода» и не узнает, что туда вводить.

Правило: каждое поле должно иметь **<label>**. Без исключений.

⚠ **placeholder** — не замена метки. Он исчезает, как только человек начинает печатать. Заполнили полформы, отвлеклись — и уже не помните, что в каком поле.

РАЗБИРАЕМСЯ

Проверка прямо в браузере

Браузер умеет проверять поля сам, без единой строчки кода:

```
<input type="email" name="email" required>
<input type="tel" name="phone" required pattern="[0-9+ ]{9,20}">
<input type="number" name="qty" min="1" max="99" required>
<input type="text" name="name" minlength="2" maxlength="50" required>
```

АТТРИБУТ	ЧТО ПРОВЕРЯЕТ
<code>required</code>	Поле обязательное, пустым не отправится
<code>min</code> / <code>max</code>	Границы для чисел
<code>minlength</code> / <code>maxlength</code>	Длина текста
<code>pattern</code>	Соответствие шаблону

Очень удобно. Но запомните намертво — мы вернёмся к этому в главе 36:

⚠ Проверка в браузере — это удобство, а не защита. Её отключают за десять секунд через F12. Все данные **обязательно** проверяются ещё раз на сервере в PHP. Всегда. Без исключений.

```
<h2>Оформление заказа</h2>

<form action="/order.php" method="POST">

    <input type="hidden" name="tovar_id" value="42">

    <p>
        <label for="name">Ваше имя</label><br>
        <input type="text" id="name" name="name" required minlength="2">
    </p>

    <p>
        <label for="phone">Телефон</label><br>
        <input type="tel" id="phone" name="phone" required placeholder="+992 _
    </p>

    <p>
        <label for="qty">Количество</label><br>
        <input type="number" id="qty" name="qty" value="1" min="1" max="20">
    </p>

    <p>
        <label for="brand">Предпочитаемый бренд</label><br>
        <select id="brand" name="brand">
            <option value="">Любой</option>
            <option value="bosch">Bosch</option>
            <option value="mann">Mann</option>
        </select>
    </p>

    <fieldset>
        <legend>Способ получения</legend>
        <input type="radio" id="d1" name="dostavka" value="samovyvoz" checked>
        <label for="d1">Самовывоз, Худжанд</label><br>
        <input type="radio" id="d2" name="dostavka" value="kurier">
        <label for="d2">Доставка курьером</label>
    </fieldset>

    <p>
```

```
<label for="comment">Комментарий</label><br>
<textarea id="comment" name="comment" rows="3"
  placeholder="Марка, год, объём двигателя"></textarea>
</p>

<p>
  <input type="checkbox" id="soglasie" name="soglasie" required>
  <label for="soglasie">Согласен на обработку данных</label>
</p>

<button type="submit">Оформить заказ</button>
</form>
```

Новый тег: `<fieldset>` группирует связанные поля, а `<legend>` даёт группе название. Полезно для радиокнопок.

Прайс-лист

Прайс-лист на тормозную систему, март 2026

Товар	Артикул	Цена
Колодки Bosch	0986424815	250 сомони
Фильтр Mann	W71280	45 сомони
Диски Brembo	09.9468.11	780 сомони

Оформление заказа

Ваше имя

Телефон

Количество

Предпочитаемый бренд

Способ получения

- ☒ Самовывоз, Худжанд
☐ Доставка курьером

Комментарий

Марка, год, объём
двигателя

☐ Согласен на обработку данных

Оформить заказ

Нажмите «Оформить заказ», не заполнив поля, — браузер сам покажет подсказку «Заполните это поле». Это работает `required`, и мы не написали ни строчки кода.

Форма пока никуда не отправляет — обработчика `/order.php` не существует. Сделаем его в главе 24.

СЛОВО

ЧТО ОЗНАЧАЕТ

<code><table></code> / <code><tr></code> / <code><th></code> / <code><td></code>	Таблица / строка / ячейка-заголовок / ячейка данных
<code>colspan</code> / <code>rowspan</code>	Объединить ячейки по горизонтали / вертикали
<code><form></code>	Конверт для данных
<code>action</code>	Куда отправлять
<code>method</code>	Как отправлять: GET или POST
<code>name</code>	Подпись на данных. Без него данные не дойдут
<code>value</code>	Значение, которое уйдёт на сервер
<code><label for="..."></code>	Подпись к полю, связана с <code>id</code>
<code>placeholder</code>	Серая подсказка внутри поля. Исчезает при вводе
<code>required</code>	Обязательное поле
<code><fieldset></code> / <code><legend></code>	Группа полей / её название
<code>type="hidden"</code>	Невидимое поле для служебных данных

ГРАБЛИ

Грабли

Забывать `name`. Поле не отправится. Проверять в первую очередь.

Разные `name` у радиокнопок. Они перестают быть группой — можно выбрать всё сразу.

`placeholder` вместо `<label>`. Подсказка исчезает при вводе, и человек забывает, что заполняет.

Верить проверке в браузере. Отключается за секунды. Проверять на сервере.

Пароль методом GET. Он окажется в адресной строке, истории и логах. Только POST.

Таблица для раскладки страницы. Ломает мобильную версию. Для раскладки — CSS.

Кнопка без `type`. Внутри формы кнопка по умолчанию отправляет её. Если кнопка нужна для другого — пишите `type="button"`.

Задача 7.1. Сделайте таблицу-прайс на пять товаров: название, артикул, бренд, цена, наличие. С шапкой `<thead>` и подписью `<caption>`.

Задача 7.2. Найдите три ошибки:

```
<form>
  <input type="text" placeholder="Имя">
  <input type="radio" name="a" value="1"> Первый
  <input type="radio" name="b" value="2"> Второй
  <input type="password" name="pass">
</form>
```

Задача 7.3. Какой `type` подойдёт лучше всего?

1. Год выпуска автомобиля.
2. Адрес электронной почты.
3. Номер телефона.
4. Согласие с условиями.
5. Выбор города из двадцати.
6. Выбор одного из трёх способов оплаты.
7. Пароль при регистрации.

Задача 7.4. Сделайте форму поиска по каталогу: строка поиска, выпадающий список брендов, поля «цена от» и «цена до», кнопка. Какой метод здесь правильный — GET или POST? Объясните почему.

Задача 7.5. Сделайте форму обратной связи: имя, телефон, тема (выпадающий список), сообщение (textarea), галочка согласия. Все поля с `<label>` и обязательные.

Задача 7.6. Откройте форму входа на любом сайте, нажмите F12 → Elements и найдите поля. Какой у формы `method`? Какие `name` у полей?

Задача 7.7. Проверьте свою форму: нажмите отправить с пустыми полями. Появилась подсказка браузера? Теперь уберите `required` у одного поля и попробуйте снова.

Ответы — в Приложении Г.

В БОЮ В бою

Найдите в этом репозитории `auth/` — там формы входа и регистрации настоящего магазина. Посмотрите: у всех `method="POST"`, у всех полей есть `name`, есть скрытые поля со служебными данными.

Одно такое скрытое поле особенно интересное — **CSRF-токен**. Это защита от подделки запроса: злоумышленник не сможет отправить форму от вашего имени с чужого сайта. Разберём его в главе 36.

ИТОГ Итог

- Таблица — для данных со строками и столбцами, не для раскладки.
- `<tr>` — строка, `<th>` — заголовок, `<td>` — данные. Столбцов в HTML нет, они получаются сами.
- Форма — конверт. `action` — куда, `method` — как.
- GET для поиска и фильтров, POST для заказов, входа, регистрации.
- `name` обязателен. Нет `name` — данные не дойдут.
- Радиокнопки одной группы имеют одинаковый `name`.
- `<label for>` для каждого поля. `placeholder` его не заменяет.
- Проверка в браузере — удобство, защита только на сервере.

Следующая глава — практическая. Соберём полноценный каталог и закрепим весь HTML.

Практика: карточка товара и каталог

ЗАЧЕМ ЭТО

Зачем эта глава

Четыре главы мы разбирали теги поодиночке. Пора собрать из них настоящую страницу.

Сегодня вы сделаете **главную страницу магазина** целиком: шапку с меню, фильтры, каталог из шести товаров, подвал с контактами. Это будет тот самый файл, который мы будем улучшать всю оставшуюся книгу — сначала оформим в CSS, потом оживим на JavaScript, потом начнём подставлять товары из базы данных.

Новых тегов почти не будет. Будет **сборка** и несколько приёмов, о которых в учебниках обычно молчат.

РАЗБИРАЕМСЯ

Сначала — план

Профессиональный приём, который экономит часы: **прежде чем писать теги, набросайте структуру словами.**

Что должно быть на главной странице магазина?

Шапка

Название магазина

Меню: Каталог, Доставка, Контакты

Строка поиска

Основная часть

Заголовок «Каталог запчастей»

Фильтры: бренд, цена от/до

Список товаров, у каждого:

фото

название

артикул

цена

наличие

кнопка «Купить»

Подвал

Адрес и телефон

Копирайт

Теперь у каждой строчки этого плана есть подходящий тег. План превращается в код почти механически:

ПЛАН	ТЕГ
Шапка	<code><header></code>
Меню	<code><nav> + + + <a></code>
Строка поиска	<code><form> + <input type="search"></code>
Основная часть	<code><main></code>
Каждый товар	<code><article></code>
Фото	<code></code>
Цена	<code><p> + </code>
Подвал	<code><footer></code>

Совет: делайте так всегда, особенно на первых порах. Гораздо легче править план из десяти строк, чем распутывать сто строк готового кода.

Начнём с одного товара — потом просто повторим.

```
<article class="tovar">
  

  <h3>Тормозные колодки Bosch</h3>

  <p class="artikul">Артикул: <code>0986424815</code></p>
  <p class="cena"><strong>250</strong> сомони</p>
  <p class="nalichie">В наличии: 7 шт.</p>

  <form action="/cart.php" method="POST">
    <input type="hidden" name="tovar_id" value="1">
    <button type="submit">В корзину</button>
  </form>
</article>
```

Что здесь важного:

Заголовок товара — `<h3>`, а не `<h1>`. Потому что `<h1>` уже занят названием магазина, `<h2>` — заголовком «Каталог». Товар — третий уровень. Иерархия соблюдена.

Классы расставлены заранее. `tovar`, `cena`, `nalichie` пока ничего не делают — CSS-то нет. Но в главе 9 мы по ним найдём элементы и оформим. Привыкайте называть части сразу.

Кнопка внутри формы. Товар в корзину — это изменение данных, значит POST. Скрытое поле передаёт, какой именно товар положить.

► 1. Комментарии в коде

```
←!— Шапка сайта —→  
<header>  
...  
</header>  
←!— /Шапка —→
```

Всё между `←!—` и `—→` браузер игнорирует. Это заметки для человека.

Зачем: когда файл вырастет до трёхсот строк, комментарии станут указателями. Особенно полезно пометать **конец** большого блока — сразу видно, какой `</div>` что закрывает.

⚠ HTML-комментарии видны в исходном коде страницы любому. Никогда не пишите в них пароли, заметки вроде «тут костыль, не трогать» или служебные адреса.

► 2. ` ` — неразрывный пробел

```
<p>Цена: 250&nbsp;сомони</p>  
<p>Телефон: +992&nbsp;92&nbsp;777&nbsp;77&nbsp;77</p>
```

Обычный пробел позволяет браузеру перенести строку. Иногда это уродливо:

Цена: 250
сомони

` ` — пробел, по которому **нельзя переносить**. Число и единица измерения останутся вместе.

Где ставить: между числом и единицей (`250 сомони` , `12 мес`), в номерах телефонов, между инициалами и фамилией.

Такие записи называются **спецсимволами** и всегда начинаются с `&`:

ЗАПИСЬ	ЧТО ДАЁТ	ЗАЧЕМ
<code>&nbsp;</code>	Неразрывный пробел	Не разрывать «250 сомони»
<code>&mdash;</code>	—	Тире по-русски
<code>&laquo;</code> <code>&raquo;</code>	« »	Кавычки-ёлочки
<code>&copy;</code>	©	Копирайт
<code>&lt;</code> <code>&gt;</code>	< >	Показать уголки как текст
<code>&</code>	&	Показать сам амперсанд

Последние три обязательны, если нужно **написать теги как текст**. Иначе браузер попытается их выполнить.

► 3. Один товар — потом копия

Не пишите шесть карточек подряд руками. Сделайте **одну и отладьте её**, а потом копируйте.

Почему это важно: если в единственной карточке ошибка, вы поправите её один раз. Если вы уже размножили карточку с ошибкой на шесть штук — править придётся шесть раз, и обязательно где-то забудете.

Это, кстати, прямая дорога к главному принципу программирования, который встретится в главе 23: **не повторяйся**. Скоро мы вообще перестанем копировать карточки — их будет создавать РНР в цикле.

Вот полный файл. Наберите его — это ваш магазин.

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Автозапчасти Firdavs — запчасти в Худжанде</title>
  <meta name="description" content="Автозапчасти для японских и немецких
    автомобилей в Худжанде. Доставка по Таджикистану.">
</head>
<body>

  <!-- Шапка -->
  <header>
    <h1>Автозапчасти Firdavs</h1>
    <p>Запчасти для японских и немецких авто &mdash; Худжанд</p>

    <nav>
      <ul>
        <li><a href="/">Главная</a></li>
        <li><a href="/catalog">Каталог</a></li>
        <li><a href="/delivery">Доставка</a></li>
        <li><a href="/contacts">Контакты</a></li>
      </ul>
    </nav>

    <form action="/search" method="GET">
      <label for="q">Поиск по каталогу</label>
      <input type="search" id="q" name="q" placeholder="Название или артикул">
      <button type="submit">Найти</button>
    </form>
  </header>
  <!-- /Шапка -->

  <main>
    <h2>Каталог запчастей</h2>

    <!-- Фильтры -->
```

```

<form action="/catalog" method="GET">
  <fieldset>
    <legend>Подобрать</legend>

    <label for="brand">Бренд</label>
    <select id="brand" name="brand">
      <option value="">Все</option>
      <option value="bosch">Bosch</option>
      <option value="mann">Mann</option>
      <option value="denso">Denso</option>
      <option value="brembo">Brembo</option>
    </select>

    <label for="min">Цена от</label>
    <input type="number" id="min" name="min" min="0" placeholder="0">

    <label for="max">до</label>
    <input type="number" id="max" name="max" min="0" placeholder="5000">

    <button type="submit">Показать</button>
  </fieldset>
</form>
<!-- /Фильтры -->

<!-- Товары -->
<article class="tovar">
  <h3>Тормозные колодки Bosch</h3>
  <p class="artikul">Артикул: <code>0986424815</code></p>
  <p class="cena"><strong>250</strong> &nbsp; сомони</p>
  <p class="nalichie">В наличии: 7 &nbsp; шт.</p>
  <form action="/cart.php" method="POST">
    <input type="hidden" name="tovar_id" value="1">
    <button type="submit">В корзину</button>
  </form>
</article>

<article class="tovar">
  <h3>Масляный фильтр Mann W71280</h3>
  <p class="artikul">Артикул: <code>W71280</code></p>
  <p class="cena"><strong>45</strong> &nbsp; сомони</p>
  <p class="nalichie">В наличии: 23 &nbsp; шт.</p>
  <form action="/cart.php" method="POST">

```

```

        <input type="hidden" name="tovar_id" value="2">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Свечи зажигания Denso</h3>
    <p class="artikul">Артикул: <code>IK20</code></p>
    <p class="cena"><strong>120</strong>&nbsp;сомони</p>
    <p class="nalichie nety">Под заказ, 5&nbsp;дней</p>
    <form action="/cart.php" method="POST">
        <input type="hidden" name="tovar_id" value="3">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Тормозные диски Brembo</h3>
    <p class="artikul">Артикул: <code>09.9468.11</code></p>
    <p class="cena"><strong>780</strong>&nbsp;сомони</p>
    <p class="nalichie">В наличии: 2&nbsp;шт.</p>
    <form action="/cart.php" method="POST">
        <input type="hidden" name="tovar_id" value="4">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Воздушный фильтр Mann C25114</h3>
    <p class="artikul">Артикул: <code>C25114</code></p>
    <p class="cena"><strong>65</strong>&nbsp;сомони</p>
    <p class="nalichie">В наличии: 14&nbsp;шт.</p>
    <form action="/cart.php" method="POST">
        <input type="hidden" name="tovar_id" value="5">
        <button type="submit">В корзину</button>
    </form>
</article>

<article class="tovar">
    <h3>Аккумулятор Bosch S4 60Ah</h3>
    <p class="artikul">Артикул: <code>0092S40050</code></p>
    <p class="cena"><strong>950</strong>&nbsp;сомони</p>

```

```

        <p class="nalichie">В наличии: 4 шт.</p>
        <form action="/cart.php" method="POST">
            <input type="hidden" name="tovar_id" value="6">
            <button type="submit">В корзину</button>
        </form>
    </article>
    <!-- /Товары -->
</main>

<!-- Подвал -->
<footer>
    <h2>Контакты</h2>
    <p>Худжанд, ул. Ленина 15<br>
        Телефон: <a href="tel:+992927777777">+992 92 777 77<br>
        Почта: <a href="mailto:info@autodoc.tj">info@autodoc.tj</a></p>

    <h3>Режим работы</h3>
    <ul>
        <li>Пн — Пт: 9:00 — 18:00</li>
        <li>Сб: 9:00 — 14:00</li>
        <li>Вс: выходной</li>
    </ul>

    <p><small>&copy; 2026 Автозапчасти Firdavs</small></p>
</footer>
<!-- /Подвал -->

</body>
</html>

```

РАЗБОР ПО СЛОВАМ

Разбор по словам

В коде появились две новые строки в `<head>`. Разберём — они обе важные.

► `<meta name="viewport" ...>`

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Это строка, без которой сайт не работает на телефоне.

Без неё мобильный браузер решает: «сайт, наверное, старый и рассчитан на компьютер». Он делает вид, что экран шириной 980 пикселей, и уменьшает всю страницу, чтобы влезла. Текст становится нечитаемым, и человек вынужден растягивать пальцами.

ЧАСТЬ

ЧТО ГОВОРIT

`width=device-width`

«Ширина страницы = настоящая ширина экрана»

`initial-scale=1`

«Не уменьшать при открытии»

Пишется в каждой странице, всегда. Мы вернёмся к ней в главе 12, когда займёмся мобильной версией всерьёз.

► `<meta name="description" ...>`

```
<meta name="description" content="Автозапчасти для японских и немецких автомоби
```

Это текст, который человек увидит в результатах поиска Google — серое описание под синей ссылкой.

Если его не написать, поисковик выдернет случайный кусок текста со страницы, и выглядеть это будет так себе. Пишите 120–160 символов, по-человечески, с пользой для читателя.

Вместе с `<title>` это два поля, которые решают, кликнет человек по вашему сайту в поиске или пролистает дальше.

► Ещё раз про иерархию заголовков

Посмотрите на нашу страницу целиком:

<code><h1></code> Автозапчасти Firdavs	← название сайта, ОДИН
<code><h2></code> Каталог запчастей	← раздел
<code><h3></code> Тормозные колодки	← товар
<code><h3></code> Масляный фильтр	← товар
<code><h2></code> Контакты	← раздел
<code><h3></code> Режим работы	← подраздел

Ровное дерево без пропусков. Именно так поисковик строит «оглавление» вашей страницы.

НА ЭКРАНЕ

На экране

Автозапчасти Firdavs

Запчасти для японских и немецких авто — Худжанд

- [Главная](#)
- [Каталог](#)
- [Доставка](#)
- [Контакты](#)

Поиск по каталогу

Каталог запчастей

Подобрать

Бренд	Все	Цена от	0	до	5000	Показать
-------	----------------------------------	---------	--------------------------------	----	-----------------------------------	---

Тормозные колодки Bosch

Артикул: 0986424815

250 сомони

В наличии: 7 шт.

Масляный фильтр Mann W71280

Артикул: W71280

45 сомони

В наличии: 23 шт.

Свечи зажигания Denso

Артикул: IK20

120 сомони

Под заказ, 5 дней

Тормозные диски Brembo

Артикул: 09.9468.11

780 сомони

В наличии: 2 шт.

Воздушный фильтр Mann C25114

Артикул: C25114

65 сомони

В наличии: 14 шт.

Аккумулятор Bosch S4 60Ah

Артикул: 0092540050

950 сомони

В наличии: 4 шт.

Контакты

Вот он, ваш магазин. Пока — длинная чёрно-белая простыня.

Давайте честно: до нормального сайта отсюда далеко. Товары идут в столбик, формы разъехались, фильтры выглядят как случайный набор полей.

Но остановитесь на секунду и посмотрите, что уже есть:

- поиск работает как поле поиска, на телефоне вызовет нужную клавиатуру;
- фильтр по бренду раскрывается списком;
- поля цены принимают только числа;
- у каждого товара своя кнопка со своим номером;
- телефон нажимается и звонит;
- структура страницы правильная и семантическая;
- поисковик увидит и заголовок, и описание.

Скелет готов и он правильный. Дальше — одежда.

И вот что приятно: в следующей главе мы не будем трогать этот файл почти совсем. Мы создадим рядом файл со стилями, подключим его одной строчкой — и та же самая страница превратится в магазин. Разделение обязанностей из первой главы — вот оно, в действии.

ГРАБЛИ

Габли

Забыть `viewport`. На телефоне сайт будет крошечным. Проверьте: откройте свою страницу на телефоне или включите F12 → значок телефона в углу панели.

Не закрыть тег в середине файла. Страница поедет начиная с этого места. Ищите так: сузьте область — прокомментируйте половину `<!-- -->`, посмотрите, осталась ли проблема, и так далее. Приём называется «деление пополам», им ищут ошибки во всём.

Забыть `alt` у картинок. В нашем примере картинок нет специально — чтобы страница работала у всех. Как только добавите свои фото, `alt` обязателен.

Скопировать карточку и забыть поменять `tovar_id`. Все шесть кнопок будут класть в корзину один и тот же товар. Классическая ошибка копирования.

Задача 8.1. Соберите эту страницу у себя целиком. Замените товары на свои — хоть на телефоны, хоть на книги, хоть на цветы. Магазин необязательно должен быть автомобильным.

Задача 8.2. Добавьте седьмой и восьмой товар. Не забудьте про `товар_id`.

Задача 8.3. Сделайте вторую страницу — `товар.html`, подробную карточку одного товара. Возьмите пример из главы 6 и добавьте: таблицу характеристик, форму заказа из главы 7, ссылку «← Назад в каталог».

Задача 8.4. Свяжите страницы: в каталоге сделайте название каждого товара ссылкой на `товар.html`. Проверьте, что переходы работают в обе стороны.

Задача 8.5. Добавьте в подвал вложенное меню каталога (`<ul>` внутри `<li>`): три раздела, в каждом по два подраздела.

Задача 8.6. Проверьте свою страницу валидатором. Откройте `validator.w3.org`, вкладка *Validate by Direct Input*, вставьте свой код и нажмите Check.

Валидатор — это проверка HTML на ошибки. Он найдёт незакрытые теги, неправильную вложенность, забытые атрибуты. Сколько ошибок у вас? Исправьте все.

Задача 8.7. Откройте страницу на телефоне (или через F12 → значок телефона). Всё читается? Теперь уберите строку `viewport`, обновите и посмотрите снова. Разницу видно?

Задача 8.8. Напишите план (словами, как в начале главы) для страницы «Доставка и оплата». Какие теги подойдут каждому пункту? Потом соберите её.

Ответы — в Приложении Г.

Вы сейчас сделали руками то, что настоящий магазин делает автоматически.

Откройте в репозитории `index.php` — там та же структура: шапка, каталог, подвал. Но карточки товаров не написаны шесть раз. Там один **шаблон карточки**, который PHP повторяет в цикле столько раз, сколько товаров пришло из базы.

Шесть товаров или шесть тысяч — кода одинаково.

Именно поэтому мы учим сначала HTML: **чтобы было что повторять**. В главе 25 вы возьмёте эту самую карточку и научите PHP размножать её самостоятельно.

ИТОГ

Итог

- Перед кодом — **план словами**. Потом каждой строке плана подбирается тег.
- Карточка товара: `<article>` + `<h3>` + данные + форма с `<input type="hidden">`.
- **Комментарии** `<!-- -->` — заметки для человека. Не пишите в них секретов.
- ` ` не даёт разорвать «250 сомони». `—`, `«`, `©` — тире, кавычки, копирайт.
- `<meta name="viewport">` обязателен, иначе сайт непригоден на телефоне.
- `<meta name="description">` — текст под ссылкой в результатах поиска.
- Иерархия заголовков без пропусков: `<h1>` → `<h2>` → `<h3>`.
- Отладьте **одну** карточку, потом копируйте. Скоро копировать перестанем совсем.
- Проверяйте себя валидатором `validator.w3.org`.

Часть II закончена. Вы умеете размечать страницы. Со следующей главы начинается CSS — и ваш магазин наконец станет красивым.



Продолжение следует.